

# **RDBMS Using Oracle**

By JJ Sir



# **Introduction to** **DBMS & RDBMS**

CH-1 DBMS Overview, SQL, SQL\*Plus

# Data and Database :

## □ What is data :

- ▣ A little information about anything is call **Data**

## □ Example

- ▣ Your name

## □ What is Database :

- ▣ Collection of meaningful data is call **Database**

## □ Example :

- ▣ Telephone directory
- ▣ Your subject books

# DBMS and RDBMS :

## □ What is DBMS :

- ▣ DBMS stands for **Database Management System**.
- ▣ DBMS used to manage your data like insert, update, delete etc

## □ What is RDBMS :

- ▣ RDBMS stands for **Relational Database Management System**
- ▣ RDBMS use to manage your data in tabular format like your data store in particular table which have some fields and each fields have some data which can be retrieved using SQL(Structure Query Language).

# Dr. E.F. Codd Rules

- ❑ Dr E.F. Codd, also known to the world as the 'Father of Database Management Systems' had propounded 12 rules which are in-fact 13 in number.
- ❑ The rules are numbered from zero to twelve. According to him, a DBMS is fully relational if it abides by all his twelve rules.

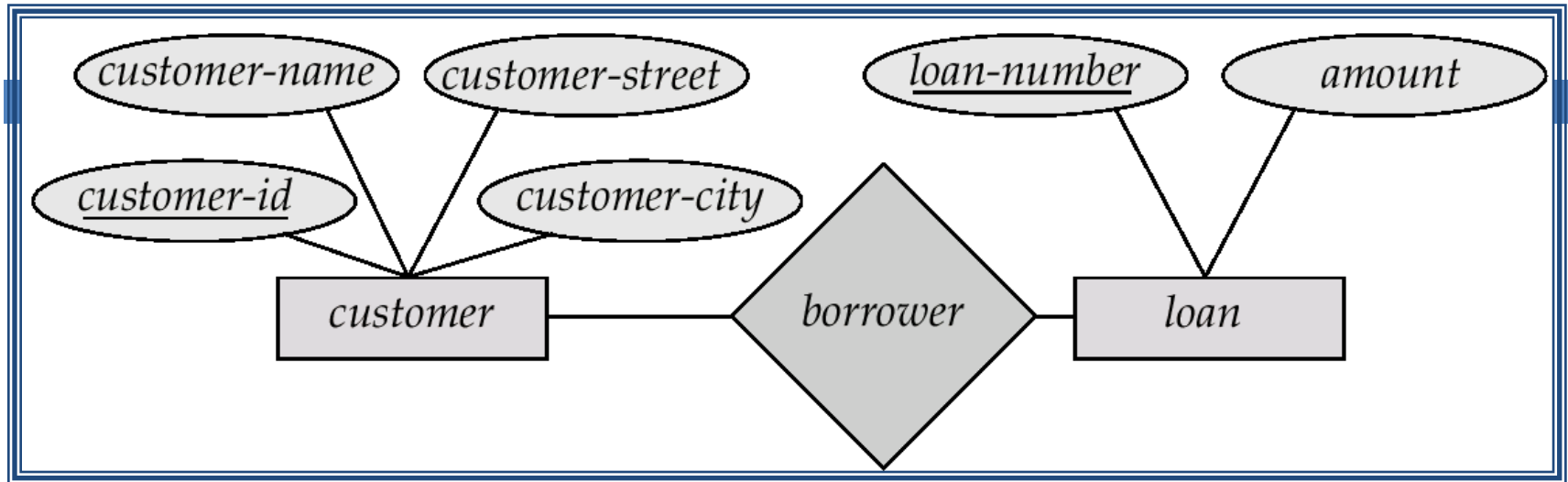
# Dr. E.F. Codd Rules

- **Rule 0 – Foundation rule**
- Any relational database management system that is propounded to be RDBMS or advocated to be a RDBMS should be able to manage the stored data in its entirety through its relational capabilities.
- **Rule 1 – Rule of Information**
- Relational Databases should store the data in the form of relations. Tables are relations in Relational Database Management Systems.
- **Rule 2 – Rule of Guaranteed Access**
- The use of pointers to access data logically is strictly forbidden. Every data entity which is atomic in nature should be accessed logically by using a right combination of the name of table, primary key represented by a specific row value and column name represented by attribute value.
- **Rule 3 – Rule of Systematic Null Value Support**
- Null values are completely supported in relational databases. They should be uniformly considered as 'missing information'. Null values are independent of any data type.
- **Rule 4 – Rule of Active and online relational Catalog**
- In the Database Management Systems lexicon, 'metadata' is the data about the database or the data about the data. The active online catalog that stores the metadata is called 'Data dictionary'.
- **Rule 5 – Rule of Comprehensive Data Sub-language**
- A single robust language should be able to define integrity constraints, views, data manipulations, transactions and authorizations.

# Dr. E.F. Codd Rules

- **Rule 6 – Rule of Updating Views**
- Views should reflect the updates of their respective base tables and vice versa. A view is a logical table which shows restricted data. Views generally make the data readable but not modifiable. Views help in data abstraction.
- **Rule 7 – Rule of Set level insertion, update and deletion**
- A single operation should be sufficient to retrieve, insert, update and delete the data.
- **Rule 8 – Rule of Physical Data Independence**
- Batch and end user operations are logically separated from physical storage and respective access methods.
- **Rule 9 – Rule of Logical Data Independence**
- Batch and end users can change the database schema without having to recreate it or recreate the applications built upon it.
- **Rule 10 – Rule of Integrity Independence**
- Integrity constraints should be available and stored as metadata in data dictionary and not in the application programs.
- **Rule 11 – Rule of Distribution Independence**
- The Data Manipulation Language of the relational system should not be concerned about the physical data storage and no alterations should be required if the physical data is centralized or distributed.
- **Rule 12 – Rule of Non Subversion**
- Any row should obey the security and integrity constraints imposed. No special privileges are applicable.

# E-R Diagrams



- **Rectangles** represent table sets.
- **Diamonds** represent relationship between tables.
- **Lines** link attributes represent connection between tables with relationship and field.
- **Ellipses** represent table fields
- **Underline** indicates primary key attributes

# TYPES OF RELATIONSHIP



# Types of Relationship in Database :-

- ❑ Relationships define how entities relate to one another.
- ❑ Types of relationship
  - ▣ One to One relationship
  - ▣ One to many relationship
  - ▣ many to one relationship
  - ▣ Many to many relationships

# One to One relationship :-

- ❑ **One to One Relationship (1:1):**
- ❑ It is used to create a relationship between two tables in which a single row of the first table can only be related to one and only one records of a second table.
- ❑ Similarly, the row of a second table can also be related to anyone row of the first table.

# One to one relationship example :-

Customer (*Table A*)

|              |
|--------------|
| Peter        |
| Susan McLain |
| Rosales      |
| Bob Jones    |
| Sue Williams |
| Brown        |
| Howard       |
| Taylor       |

Order (*Table B*)

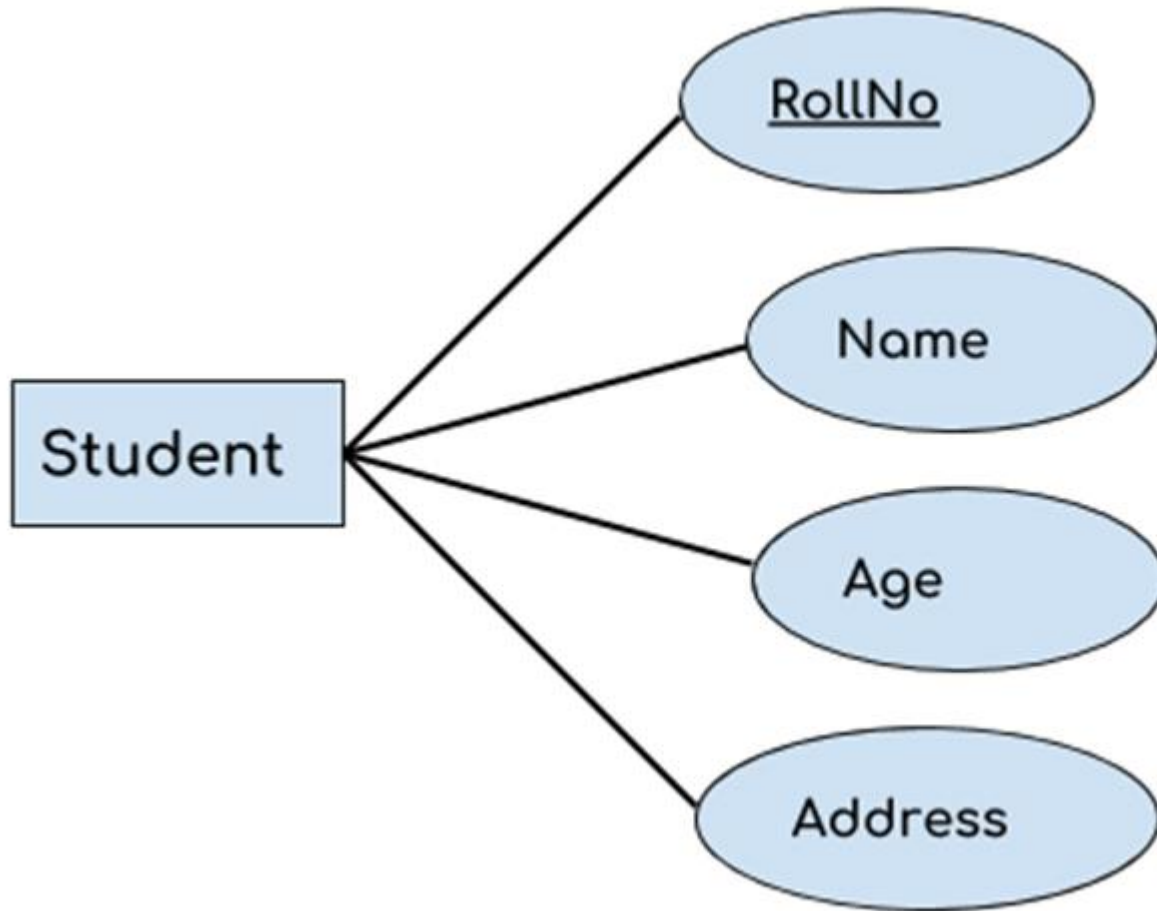
|        |
|--------|
| Ord102 |
| Ord103 |
| Ord105 |
| Ord107 |
| Ord108 |
| Ord120 |
| Ord106 |
| Ord100 |



# One to Many Relationship

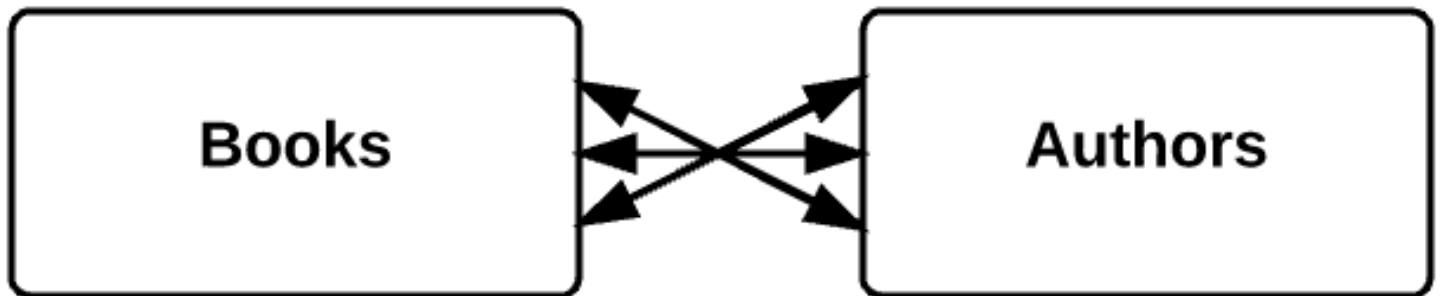
- ❑ **One to Many Relationship:** It is used to create a relationship between two tables. Any single rows of the first table can be related to one or more rows of the second tables, but the rows of second tables can only relate to the only row in the first table.
- ❑ It is also known as a **many to one** relationship.

# One to Many Relationship example :-



# Many to Many Relationship:

- ❑ **Many to Many Relationship:**
- ❑ It is **many to many** relationships that create a relationship between two tables.
- ❑ Each record of the first table can relate to any records (or no records) in the second table.  
Similarly, each record of the second table can also relate to more than one record of the first table.
- ❑ It is also represented as an **N:N** relationship.



# NORMALIZATION



# What is Normalization?



- Normalization is a database design technique which organizes tables in a manner that reduces redundancy and dependency of data.
- It divides larger tables to smaller tables and links them using relationships.

# Not normal table

| Full Names  | Physical Address          | Movies rented                                     | Salutation | Category            |
|-------------|---------------------------|---------------------------------------------------|------------|---------------------|
| Janet Jones | First Street Plot No 4    | Pirates of the Caribbean,<br>Clash of the Titans  | Ms.        | Action, Action      |
| Robert Phil | 3 <sup>rd</sup> Street 34 | Forgetting Sarah Marshal,<br>Daddy's Little Girls | Mr.        | Romance,<br>Romance |
| Robert Phil | 5 <sup>th</sup> Avenue    | Clash of the Titans                               | Mr.        | Action              |

# Database Normal Forms

- ❑ **1NF (First Normal Form) Rules**
- ❑ Each table cell should contain a single value.
- ❑ Each record needs to be unique.

# 1 NF solution

| FULL NAMES  | PHYSICAL ADDRESS          | MOVIES RENTED            | SALUTATION |
|-------------|---------------------------|--------------------------|------------|
| Janet Jones | First Street Plot No 4    | Pirates of the Caribbean | Ms.        |
| Janet Jones | First Street Plot No 4    | Clash of the Titans      | Ms.        |
| Robert Phil | 3 <sup>rd</sup> Street 34 | Forgetting Sarah Marshal | Mr.        |
| Robert Phil | 3 <sup>rd</sup> Street 34 | Daddy's Little Girls     | Mr.        |
| Robert Phil | 5 <sup>th</sup> Avenue    | Clash of the Titans      | Mr.        |

# 2NF (Second Normal Form) Rules

---

- Rule 1 - Be in 1NF
- Rule 2- Single Column Primary Key

# 1nf

| FULL NAMES  | PHYSICAL ADDRESS          | MOVIES RENTED            | SALUTATION |
|-------------|---------------------------|--------------------------|------------|
| Janet Jones | First Street Plot No 4    | Pirates of the Caribbean | Ms.        |
| Janet Jones | First Street Plot No 4    | Clash of the Titans      | Ms.        |
| Robert Phil | 3 <sup>rd</sup> Street 34 | Forgetting Sarah Marshal | Mr.        |
| Robert Phil | 3 <sup>rd</sup> Street 34 | Daddy's Little Girls     | Mr.        |
| Robert Phil | 5 <sup>th</sup> Avenue    | Clash of the Titans      | Mr.        |

# 2NF solution

| MEMBERSHIP ID | FULL NAMES  | PHYSICAL ADDRESS          | SALUTATION |
|---------------|-------------|---------------------------|------------|
| 1             | Janet Jones | First Street Plot No 4    | Ms.        |
| 2             | Robert Phil | 3 <sup>rd</sup> Street 34 | Mr.        |
| 3             | Robert Phil | 5 <sup>th</sup> Avenue    | Mr.        |

| MEMBERSHIP ID | MOVIES RENTED            |
|---------------|--------------------------|
| 1             | Pirates of the Caribbean |
| 1             | Clash of the Titans      |
| 2             | Forgetting Sarah Marshal |
| 2             | Daddy's Little Girls     |
| 3             | Clash of the Titans      |

# What are transitive functional dependencies?

- A transitive functional dependency is when changing a non-key column, might cause any of the other non-key columns to change

| MEMBERSHIP ID | FULL NAMES  | PHYSICAL ADDRESS          | SALUTATION |
|---------------|-------------|---------------------------|------------|
| 1             | Janet Jones | First Street Plot No 4    | Ms.        |
| 2             | Robert Phil | 3 <sup>rd</sup> Street 34 | Mr.        |
| 3             | Robert Phil | 5 <sup>th</sup> Avenue    | Mr.        |

*Change in Name*

*May Change  
Salutation*

# 3NF (Third Normal Form) Rules

- Rule 1 - Be in 2NF
- Rule 2- Has no transitive functional dependencies
- To move our 2NF table into 3NF, we again need to again divide our table.

| MEMBERSHIP ID | FULL NAMES  | PHYSICAL ADDRESS          | SALUTATION ID |
|---------------|-------------|---------------------------|---------------|
| 1             | Janet Jones | First Street Plot No 4    | 2             |
| 2             | Robert Phil | 3 <sup>rd</sup> Street 34 | 1             |
| 3             | Robert Phil | 5 <sup>th</sup> Avenue    | 1             |

| MEMBERSHIP ID | MOVIES RENTED            |
|---------------|--------------------------|
| 1             | Pirates of the Caribbean |
| 1             | Clash of the Titans      |
| 2             | Forgetting Sarah Marshal |
| 2             | Daddy's Little Girls     |
| 3             | Clash of the Titans      |

| SALUTATION ID | SALUTATION |
|---------------|------------|
| 1             | Mr.        |
| 2             | Ms.        |
| 3             | Mrs.       |
| 4             | Dr.        |

- 
- There are no transitive functional dependencies, and hence our table is in 3NF
  - In Table 3 Salutation ID is primary key, and in Table 1 Salutation ID is foreign to primary key in Table 3

# Introduction to SQL

## ❑ **What is SQL?**

- ❑ Sql is known as language of database.
- ❑ Sql is stand for structured query language.
- ❑ Sql is used to communicate with data base.
- ❑ Sql developed by ibm in 1970.
- ❑ It is standard language used to interact with modern data base.
- ❑ Sql is an english like language and it has simple command set which is easy to remember.
- ❑ Sql statement are used to perform task such as insert, update and delete the data.
- ❑ Sql can used by numbers of user, including those with little or no programming experience.

# SQL COMMANDS AND DATA TYPES



# Sql command

- ❑ **There are four types of SQL command.**
- ❑ DDL (Data Definition Language)
- ❑ DML (Data Manipulation Language)
- ❑ DCL (Data Control Language)
- ❑ DQL (Data Query Language)

# Sql command

- ❑ **DDL (Data Definition Language)**

- ❑ **CREATE:** It is used to create objects (table, view, function, procedure, trigger etc.) in the database.
- ❑ **ALTER:** It is used to change the structure of database.
- ❑ **DROP:** It is used to delete objects from the database.
- ❑ **TRUNCATE:** It is used to remove all records from the table, including all spaces allocated for the records are removed.
- ❑ **GRANT:** It is used to give different command grant (access rights) to the user.
- ❑ **REVOKE:** It is used to take the given grant back from the user.

- ❑ **DML (Data Manipulation Language)**

- ❑ **INSERT:** it used to insert the data into a table
- ❑ **UPDATE:** it used to update the existing data within a table.
- ❑ **DELETE:** it used to delete all the records from the table, the space will be remaining.
- ❑ **CALL:** it is used to call a PL/SQL program.

# Sql command

- ❑ **DCL (Data Control Language)**
- ❑ **COMMIT:** it is used to save the work.
- ❑ **SAVEPOINT:** it is used to identify a point in transaction.
- ❑ **ROLLBACK:** Restore database to original since the last commit (like undo).
  
- ❑ **DQL (Data Query Language)**
- ❑ **SELECT:** It is used to retrieve data from the database.

# Data Types :-

- **char**
  - size 1 to maximum 2000 characters and if data is short than its specified than space padded rest of size, it means CHAR is used for fixed-length strings.
- **Varchar**
- **Varchar2**
  - Both varchar and varchar2 are same, it stores alphanumeric data and also store data in its size if data is short than is length than varchar2 remove extra length of data type and allocates to other data, you can say VARCHAR2 is used for variable-length strings.
- **Long**
  - Long datatype is Large Object(LOB) data types. It stored up to 2GB length. But restriction of this type is you cannot use WHERE clause, Group By clause, Order By clause, Distinct operator in SELECT statement
- **Number**
  - This data type used to store negative and positive integers: fixed-point and floating numbers with a precisions of up to 38 digits.
- **Date**
  - The date data type is used to store date and time information. Default date format is DD-MON-YY.

# Introduction to SQL\*Plus

- ❑ **What is Sql \* plus ?**
- ❑ Sql \* plus is oracle product.
- ❑ It provide command window, where sql command can be written by the user.
- ❑ SQL\*Plus is a command line interface to the Oracle Database.
- ❑ From SQL\*Plus, one can issue SQL, PL/SQL and special SQL\*Plus commands.
- ❑ SQL\*Plus provides a user-friendly interface which allows you to define and manipulate data in an Oracle database.

# SQL\*Plus formatting commands

- ❑ **SPOOL <filename>**: This allows saving SQL statements together with their outputs to a file.
- ❑ **SAVE <filename>**: This allows you to save buffer contents into a file.
- ❑ **DESC <tablename>**: Lists the attributes of tables and other objects. The command also can be written as DESCRIBE <tablename>
- ❑ **EDIT [filename]**: Places you in an editor so you can edit a SQL command. The command also can be written as ED [filename]

# SQL\*Plus formatting commands

- ❑ **EXIT or QUIT** : Disconnect from the database and terminate SQL\*Plus.
- ❑ **SET SERVEROUTPUT {OFF | ON}**: Whether to display the output of stored procedures (or PL/SQL blocks).
- ❑ **SET AUTOCOMMIT {OFF | ON}**: Commits after each SQL command or PL/SQL block automatically.
- ❑ **SET COLSEP { | text }** : The text to be printed between columns retrieved by SELECT statement. Columns normally separated by a space.

# Operator and Expression

- An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulation.
- PL/SQL language is rich in built-in operators and provides the following types of operators
  - **Comparison operators**
  - **Arithmetic operators**
  - **Relational operators**
  - **Logical operators**

# Comparison operators

---

- Three types of Comparison operators
  - ▣ IN
  - ▣ Between
  - ▣ Like

# IN operator

- The IN operator is used when you want to compare a column with more than one value. It is similar to an OR condition.
- Example
  - ▣ `select *From emp`
  - ▣ `where deptno in(10,20);`

# Between...And operator

- ❑ The operator BETWEEN and AND, are used to compare data for a range of values.
- ❑ This operator also known as range operator.
- ❑ Example
  - ❑ `select *From emp`
  - ❑ `where sal between 800 and 1500;`

# Like operator

- ❑ This operator is known as Pattern Matching operator
- ❑ The LIKE operator is used to list all rows in a table whose column values match a specified pattern.
- ❑ It is useful when you want to search rows to match a specific pattern, or when you do not know the entire value

# Like operator

- ❑ How to use like operator
- ❑ You can search by following
  - ❑ \_\_\_ this is use when you know some letters of specified search
    - ▣ SELECT \*FROM EMP
    - ▣ WHERE ENAME LIKE 'SM\_TH';
  - ❑ % this is use when you know some letters and rest of anything
    - ▣ SELECT \*FROM EMP
    - ▣ WHERE ENAME LIKE 'S%';
  - ❑ %search% this is use when you search your character anywhere
    - ▣ SELECT \*FROM EMP
    - ▣ WHERE ENAME LIKE '%S%';

# Arithmetic Operators

| Operator | Description                                                       | Example                 |
|----------|-------------------------------------------------------------------|-------------------------|
| +        | Adds two operands                                                 | A + B will give 15      |
| -        | Subtracts second operand from the first                           | A - B will give 5       |
| *        | Multiplies both operands                                          | A * B will give 50      |
| /        | Divides numerator by denominator                                  | A / B will give 2       |
| **       | Exponentiation operator, raises one operand to the power of other | A ** B will give 100000 |

# Relational Operators

| Operator       | Description                                                                                                                     | Example               |
|----------------|---------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| =              | Checks if the values of two operands are equal or not, if yes then condition becomes true.                                      | (A = B) is not true.  |
| !=<br><><br>~= | Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.                     | (A != B) is true.     |
| >              | Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.             | (A > B) is not true.  |
| <              | Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.                | (A < B) is true.      |
| >=             | Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true. | (A >= B) is not true. |
| <=             | Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.    | (A <= B) is true      |

# Logical Operators

| Operator | Description                                                                                                                                             | Examples               |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| and      | Called the logical AND operator. If both the operands are true then condition becomes true.                                                             | (A and B) is false.    |
| or       | Called the logical OR Operator. If any of the two operands is true then condition becomes true.                                                         | (A or B) is true.      |
| not      | Called the logical NOT Operator. Used to reverse the logical state of its operand. If a condition is true then Logical NOT operator will make it false. | not (A and B) is true. |

# SQL VS SQL \*PLUS

| SQL                                                                       | SQL *PLUS                                                                                   |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| SQL is a language for communicating with the oracle server to access data | SQL *PLUS recognizes SQL statements and sends them to Server                                |
| SQL is based on American National Standards Institute(ANSI)               | SQL *PLUS is the Oracle proprietary interface for execution SQL statements                  |
| SQL manipulates data and table definition in the database                 | SQL *PLUS does not allow manipulation of values in the database                             |
| SQL is entered into the SQL buffer on one or more lines                   | SQL *PLUS is entered one line at a time, not stored in the SQL buffer                       |
| SQL does not have continuation character                                  | SQL *PLUS uses a dash(-) as a continuation character if the command is longer than one line |
| SQL uses a termination character to execute commands immediately          | SQL *PLUS does not require termination characters ; executes commands immediately           |
| SQL uses functions to perform some formatting                             | SQL *PLUS uses commands to format data                                                      |

| DBMS                                                                  | RDBMS                                                                                                  |
|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| RELATIONSHIP BETWEEN TWO TABLES OR FILES ARE MAINTAINED PROGRAMICALLY | RELATIONSHIP BETWEEN TWO TABLES OR FILE CAN BE SPECIFIED AT THE TIME OF TABLE CREATION                 |
| DOES NOT SUPPORT CLIENT /SERVER ARCHITECTURE                          | SUPPORTS CLIENT/SERVER ARCHITECTURE                                                                    |
| DOES NOT SUPPORT DISTRIBUTED DATABSES                                 | SUPPORTS DISTRIBUTED DATABASES                                                                         |
| NO SECURITY OF DATA                                                   | MULTIPLE SECURITY LEVES;<br>1 LOGIN AT OS LEVEL<br>2 COMMAND LEVEL SECURITY<br>3 OBJECT LEVEL SECURITY |
| EACH TABLE IS GIVEN EXTENSION                                         | MANY TABLES ARE GROUPED IN ONE DATABASE IN RDBMS                                                       |
| Example :Foxpro                                                       | Example : Oracle                                                                                       |



# **Creating, Altering & Dropping Tables**

# Naming rules for table creation :-

---

- Up to 30 character long names are allowed
- Only Alphabets, Numbers and \_(underscore) are allowed
- Name should begin with Alphabet
- SQL reserve words are not allowed
- If names are defined in double quotes(""), it will be case-sensitive

# Create table :-

---

- ❑ To create table you have following information
  - ❑ Table name
  - ❑ Table fields datatype(size)
  - ❑ Each field name separated by comma (,)

# ALTER STATEMENT IN SQL

Adding, modifying, rename and dropping columns  
in a table also rename table

## □ What is the ALTER statement?

- ▣ The ALTER statement is used to modify the structure of a table in SQL.
- ▣ It can be used to add, modify, rename or drop columns in a table and also rename table.
- ▣ It can also be used to add or drop constraints on a table.

## □ Syntax of the ALTER TABLE statement:

- ▣ ALTER TABLE table\_name
- ▣ [ADD | MODIFY | DROP] column\_name column\_definition
- ▣ [, column\_name column\_definition]...;

- ❑ -- Add a new column to the table
- ❑ ALTER TABLE table\_name ADD column\_name column\_definition;
  
- ❑ -- Modify the definition of an existing column
- ❑ ALTER TABLE table\_name MODIFY column\_name column\_definition;
  
- ❑ --Rename column of an existing column
- ❑ ALTER TABLE table\_name Rename COLUMN column\_name to newname;
  
- ❑ --Rename table name
- ❑ ALTER TABLE table\_name Rename to new\_tablename;
  
- ❑ -- Drop a column from the table
- ❑ ALTER TABLE table\_name DROP column\_name;

- ❑ Constraints that can be added or dropped with the ALTER TABLE statement:
  - ▣ Primary key constraints
  - ▣ Foreign key constraints
  - ▣ Check constraints
  - ▣ Unique constraints
- ❑ Limitations of the ALTER TABLE statement:
  - ▣ The ALTER TABLE statement cannot be used to change the data type of a column if there is data in the column.

# DROPPING TABLE

- ❑ The DROP TABLE statement is used to drop an existing table in a database.
- ❑ **Syntax:-**
- ❑ DROP TABLE *table\_name*;

DATA MANIPULATION  
COMMAND LIKE INSERT,  
UPDATE, DELETE

# Data Manipulation Command

- ❑ Data manipulation commands are used to retrieve, modify, or manipulate data within a database or data storage system.
- ❑ The specific command you use depends on the database management system (DBMS) you are working with.
- ❑ Commands like:
  - ❑ **INSERT**
  - ❑ **UPDATE**
  - ❑ **DELETE**

# INSERT, UPDATE

- ❑ **INSERT:** Used to add new records to a table.
- ❑ **Syntax:-**
- ❑ `INSERT INTO table_name (column1, column2)  
VALUES (value1, value2);`
  
- ❑ **UPDATE:** Used to modify existing records in a table.
- ❑ **Syntax:-**
- ❑ `UPDATE table_name  
SET column1 = new_value1  
WHERE condition;`

# DELETE

- ❑ **DELETE:** Used to remove records from a table.
- ❑ **Syntax:-**
- ❑ `DELETE FROM table_name`  
`WHERE condition;`

# CONSTRAINTS



# Constraints :-

- ❑ Constraints are set of rules used to enforce data integrity.
- ❑ They don't take space in database like data.
- ❑ Even if single column fails to satisfy constrain, entire record will be rejected by Oracle.
- ❑ Types of constraints are :-
  - ▣ Primary key
  - ▣ Foreign key
  - ▣ Unique
  - ▣ NOT NULL
  - ▣ Check

## Feature of Primary key :-

- ❑ Does not allow duplicate values
- ❑ NULL values not allowed
- ❑ Primary key is not compulsory but recommended
- ❑ Primary key can not be LONG or LONG RAW data type
- ❑ Only one key is allowed per table
- ❑ Unique Index created automatically on Primary key

# Define primary key :-

- ---primary key at column level----

- create table uni1
- (ccode number(3) primary key,
- corsename varchar2(15),
- duration number(1));

- ---primary key at table level---

- create table uni
- (ccode number(3),
- corsename varchar2(15),
- duration number(1),
- primary key(ccode,corsename));

# Feature of foreign key :-

- ❑ It references a column of another table and it can be same table also
- ❑ Referenced field must be Unique or Primary key. The default is primary key
- ❑ Defined foreign key table is called child table where as defined primary key table is called parent table
- ❑ Child table may have duplicates and NULLS in foreign key column
- ❑ Parent record can be deleted only if child record does not exist
- ❑ Parent table can not be updated if child record exist

## ❑ ON DELETE CASCADE :-

- ❑ When the record is deleted in Parent table, all records in Child Table will be deleted automatically.

## ❑ ON DELETE SET NULL :-

- ❑ When record is deleted in Parent table, all field value held by foreign key of child table is set to NULL.
- ❑ Entire record will not be deleted.

# Define foreign key :-

- ❑ create table college
- ❑ (rno number(3),
- ❑ snam char(20),
- ❑ course\_no number(3) references uni1(ccode));
- ❑ on delete cascade
- ❑ on delete set null

# UNIQUE

- The purpose of UNIQUE constraint is to ensure that the data in the column is unique and it must not be repeated across the column.
- A table can have more than one UNIQUE key column.
- **UNIQUE constraint at column level:**
  - **Syntax:** Column name data type (size) UNIQUE,
- **UNIQUE constraint at table level:**
  - **Syntax:** UNIQUE (columnname [, columnname , ... ])

# NOT NULL

- ❑ A NULL value is different from space or zero.
- ❑ A NULL value can be inserted into the column of any data type.
- ❑ When a column is defined as not null, it becomes mandatory column.
- ❑ It means that a value must be entered into the column.
- ❑ **NOT NULL constraint at Column Level:**
- ❑ **Syntax:** Columnname datatype(size) NOT NULL

# CHECK

- ❑ Business Rule validations can be applied to a column using CHECK constraint.
- ❑ CHECK constraint must be specified as logical expression that evaluates either to TRUE or FALSE.
- ❑ **CHECK constraint at column level:**
  - ❑ **Syntax:** Column name data type (size) CHECK (logical expression)
- ❑ **CHECK constraint at table level:**
  - ❑ **Syntax:** CHECK (logical expression)

# SELECT STATEMENT IN ORACLE



# What is the SELECT statement?

- The SELECT statement is a SQL statement that is used to retrieve data from a database. It is one of the most commonly used SQL statement
- The syntax for a SELECT statement is as follows:
  - ▣ SELECT column\_list FROM table\_name
  - ▣ [WHERE condition]
  - ▣ [ORDER BY column\_name]



- ❑ **Column list**

- ❑ The `column_list` specifies the columns that you want to retrieve from the table. The `*` in the `column_list` indicates that all of the columns from the table should be retrieved.

- ❑ **Table name**

- ❑ The `table_name` specifies the name of the table that you want to retrieve data from.

- ❑ **WHERE condition**

- ❑ The `WHERE` clause is used to filter the rows that are returned. The condition can be a simple comparison, such as `country = 'USA'`, or it can be a more complex expression.



- ❑ **ORDER BY clause**

- ❑ The ORDER BY clause is used to sort the rows that are returned. The column name that you specify in the ORDER BY clause determines the sort order. The default sort order is ascending, but you can also specify descending order by adding the DESC keyword after the column name.

# GROUP BY AND HAVING CLAUSE, ROLLUP AND CUBE



# The SQL GROUP BY Statement

- ❑ The GROUP BY statement groups rows that have the same values into summary rows, like "find the maxsal from each job in emp table".
- ❑ The GROUP BY statement is often used with aggregate functions (COUNT, MAX, MIN, SUM, AVG) to group the result-set by one or more columns.
  - ▣ By default, GROUP BY sorts the rows in ascending order based on the values in the group column.
- ❑ GROUP BY Syntax
  - ▣ `SELECT column_name`  
`FROM table_name`  
`GROUP BY column_name`  
`ORDER BY column_name;`

## □ The SQL HAVING Clause

- Having clause special use for give condition to group by like WHERE.
- You use the HAVING clause to filter groups of rows
- GROUP BY can be used without HAVING.
- But HAVING must be used with GROUP BY.
- You place the HAVING clause after your GROUP BY clause:

## □ HAVING Syntax

- ▣ SELECT *column\_name*  
FROM *table\_name*  
GROUP BY *column\_name*  
HAVING *condition*  
ORDER BY *column\_name*;

# ROLLUP and CUBE

- ❑ ROLLUP and CUBE are **simple extensions to the `SELECT` statement's `GROUP BY` clause.**
- ❑ ROLLUP creates subtotals at any level of aggregation needed, from the most detailed up to a grand total. D
- ❑ CUBE is an extension similar to ROLLUP , enabling a single statement to calculate all possible combinations of subtotals

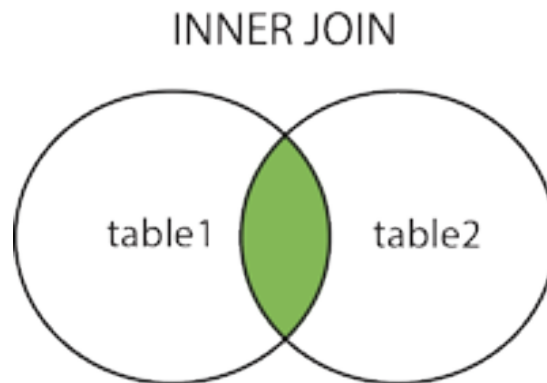
# JOINS



- ❑ SQL JOIN
- ❑ A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
- ❑ Different Types of SQL JOINS
  - ❑ Inner Join
  - ❑ Outer Join
    - ▣ Left outer Join
    - ▣ Right outer Join
    - ▣ Full outer Join
  - ❑ Self Join
  - ❑ Cross Join

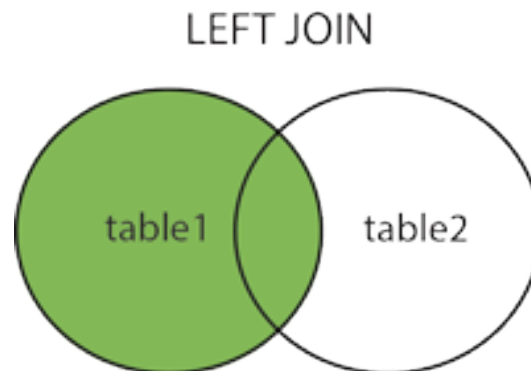
# Inner Join

- ❑ SQL INNER JOIN Keyword
- ❑ The INNER JOIN keyword selects records that have matching values in both tables.
- ❑ This join is also known as equi join

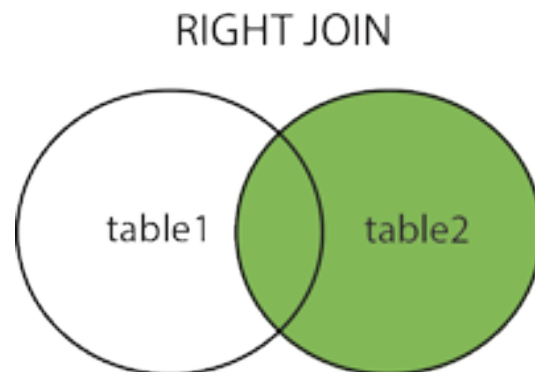


# Outer Join

- ❑ SQL LEFT JOIN Keyword
- ❑ The LEFT JOIN keyword returns all records from the left table (table1), and the matched records from the right table (table2).



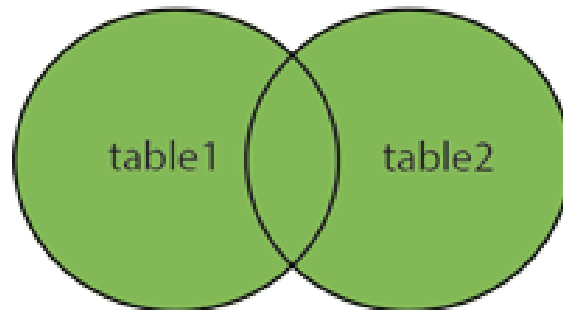
- ❑ SQL RIGHT JOIN Keyword
- ❑ The RIGHT JOIN keyword returns all records from the right table (table2), and the matched records from the left table (table1).



# Full outer join

- ❑ SQL FULL OUTER JOIN Keyword
- ❑ The FULL OUTER JOIN keyword return all records when there is a match in either left (table1) or right (table2) table records.

FULL OUTER JOIN



# Self join and cross join

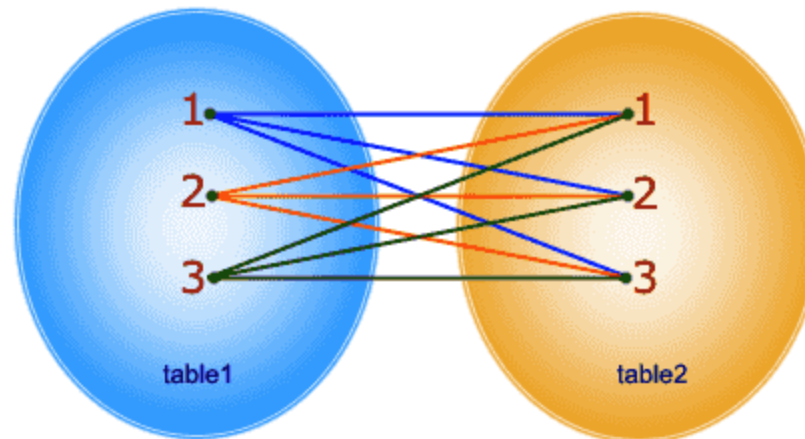
## □ SQL Self JOIN

- A self JOIN is a regular join, but the table is joined with itself.

## □ Cross join

- In this join there is no condition given so records multiply by each other table
- Like if one table have 3 records and other table have 4 and we apply cross join than result will be 12 rows display

```
SELECT * FROM table1 CROSS JOIN table2;
```



In CROSS JOIN, each row from 1st table joins with all the rows of another table.  
If 1st table contain x rows and y rows in 2nd one the result set will be  $x * y$  rows.

# SUB-QUERY



# What is subquery in SQL?

- A subquery is a SQL query nested inside a larger query.

- Syntax :

```
SELECT    select_list
FROM      table
WHERE     expr operator
```

```
(SELECT    select_list
FROM      table);
```

- The subquery (inner query) executes once before the main query (outer query) executes.
- The main query (outer query) use the subquery result.

- ❑ A sub-query answers the queries that have multiple parts
- ❑ The sub-query answers one part of the question and the parent query answers the other part.
- ❑ Types of sub-query
  - ▣ Single row sub queries
  - ▣ Multiple value sub queries
  - ▣ Multiple column sub queries
  - ▣ Multiple row sub queries

# Single row sub query :

- ❑ It returns single value from a table.
- ❑ Example
  - ❑ Select ename, sal from emp
  - ❑ Where sal = (select max(sal) from emp);
- ❑ Output
  - ❑ ENAME                      SAL
  - ❑ -----
  - ❑ KING                      5000

# Multiple value sub query :

- ❑ This returns more than one value to the parent query statement.
- ❑ IN operator is the most commonly used in multiple row sub query

- ❑ Example

- ❑ Select ename, sal, job from emp
  - ❑ Where sal in(select max(sal) from emp group by job);

- ❑ **Out put**

| ❑ ENAME  | SAL JOB        |
|----------|----------------|
| ❑ -----  | -----          |
| ❑ MILLER | 1 300 CLERK    |
| ❑ ALLEN  | 1 600 SALESMAN |
| ❑ KING   | 5000 PRESIDENT |
| ❑ JONES  | 2975 MANAGER   |
| ❑ FORD   | 3000 ANALYST   |
| ❑ SCOTT  | 3000 ANALYST   |

# Multiple column sub-query:

- ❑ This returns values more than one column in subquery
- ❑ Example
  - ❑ Select ename, sal , job from emp
  - ❑ Where (sal,job) in (select max(sal),job from emp group by job);

## ❑ **Output :**

| ❑ ENAME  | SAL   | JOB       |
|----------|-------|-----------|
| ❑ -----  | ----- | -----     |
| ❑ MILLER | 1300  | CLERK     |
| ❑ ALLEN  | 1600  | SALESMAN  |
| ❑ KING   | 5000  | PRESIDENT |
| ❑ JONES  | 2975  | MANAGER   |
| ❑ FORD   | 3000  | ANALYST   |
| ❑ SCOTT  | 3000  | ANALYST   |

# Multiple row sub-query:

- ❑ This type of sub-query returns more than one row of result from the sub-query
- ❑ Example
  - Select ename, deptno from emp
  - Where deptno not in (select deptno from emp where ename='SCOTT');

## ❑ Output

| ❑ ENAME  | DEPTNO |
|----------|--------|
| ❑ -----  |        |
| ❑ ALLEN  | 30     |
| ❑ WARD   | 30     |
| ❑ MARTIN | 30     |
| ❑ BLAKE  | 30     |
| ❑ CLARK  | 10     |
| ❑ KING   | 10     |
| ❑ TURNER | 30     |
| ❑ JAMES  | 30     |
| ❑ MILLER | 10     |

# SET OPERATORS :-

# SQL Set Operation

- The SQL Set operation is used to combine the two or more SQL SELECT statements.
  
- Types of Set Operation
  - ▣ Union
  - ▣ UnionAll
  - ▣ Intersect
  - ▣ Minus

# 1. Union

- The SQL Union operation is used to combine the result of two or more SQL SELECT queries.
- The union operation eliminates the duplicate rows from its result set.
- **Syntax**
  - ▣ SELECT column\_name FROM table1
  - ▣ UNION
  - ▣ SELECT column\_name FROM table2;
- 
- **EXAMPLE**
  - ▣ select \*from emp where deptno = 10
  - ▣ union
  - ▣ select \*from emp where deptno in (10,20);

## 2. Union All

- Union All operation is equal to the Union operation. It returns the set without removing duplication and sorting the data.
- **Syntax:**
  - ▣ SELECT column\_name FROM table1
  - ▣ UNION ALL
  - ▣ SELECT column\_name FROM table2;
- 
- **EXAMPLE**
  - ▣ select \*from emp where deptno = 10
  - ▣ union all
  - ▣ select \*from emp where deptno in (10,20);

# 3. Intersect

- It is used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
- **Syntax**
  - ▣ SELECT column\_name FROM table1
  - ▣ INTERSECT
  - ▣ SELECT column\_name FROM table2;
- **EXAMPLE**
  - ▣ select \*from emp where deptno in (10,20)
  - ▣ intersect
  - ▣ select \*from emp where deptno in (20,30);

# 4. Minus

- ❑ It combines the result of two SELECT statements. Minus operator is used to display the rows which are present in the first query but absent in the second query.
- ❑ **Syntax:**
  - ❑ SELECT column\_name FROM table1
  - ❑ MINUS
  - ❑ SELECT column\_name FROM table2;
  - ❑
- ❑ **Example**
  - ❑ select \*from emp where deptno in (10,20)
  - ❑ minus
  - ❑ select \*from emp where deptno = 20;

# LIBRARY FUNCTIONS :-



# Aggregate function

- ❑ AVG : returns average value from table
  - ▣ Select avg(sal) from emp
- ❑ MIN : returns minimum value from table
  - ▣ Select min(sal) from emp
- ❑ MAX : returns maximum value from table
  - ▣ Select max(sal) from emp
- ❑ SUM : return sum value
  - ▣ Select sum(sal) from emp

# Numeric Function

- ABS : returns absolute value
  - ▣ Select abs(-12) from dual;
- Power : return power of given value
  - ▣ Select power(2,3) from dual
- Round : return rounded value
  - ▣ Select round(12.3273,2) from dual;
- Sqrt : returns square root value
  - ▣ Select sqrt(100) from dual
- Greatest : returns the biggest value out of list of nos.
  - ▣ Select greatest(10,20,30) from dual;
- Least : returns smallest value out of list of nos.
  - ▣ Select least(10,20,30) from dual;

# Numeric Function

- ❑ MIN : returns minimum value from given number
  - ❑ Select min(10,20,30) from dual;
- ❑ MAX : returns maximum value from given number
  - ❑ Select max(10,20,30) from dual;
- ❑ Mod : returns reminder after dividing 1<sup>st</sup> no. from 2nd no.
  - ❑ Select mod(20,8) from dual;
- ❑ Trunc : truncates no. for specified no. of decimal place and ignore math rule.
  - ❑ Select trunc(12.3489,2) from dual;
- ❑ Floor : returns rounded nearest value of given number
  - ❑ Select floor(12.55) from dual
  - ❑ Output : 12
- ❑ Ceil : returns nearest big value of the given number
  - ❑ Select ceil(21.55) from dual
  - ❑ Output : 22

# Character Function

- ❑ Lower : converts all capitals into small
- ❑ Upper : converts all small into capital
- ❑ Initcap : converts first character of all words into capital and remaining into small
  - ▣ Select initcap('welcome to hjd college') from dual;
- ❑ Substr : returns part of string
  - ▣ Select substr('hello india',3,3) from dual;

# Character Function

- ❑ Ascii : returns ascii code of first character of string
  - ▣ Select ascii('oracle') from dual
- ❑ Instr : finds the position of one string into another
  - ▣ Select instr('I like oracle','oracle') from dual;
- ❑ Translate : replace character in string with another
  - ▣ Select translate('I am back','abc','xyz') from dual;
- ❑ Length : returns length of string
  - ▣ Select length('hellow world') from dual

# Character Function

- ❑ Ltrim :
  - ▣ All the characters are removed from left side as long as they appear in given set
  - ▣ Select ltrim('I like India','I li') from dual
- ❑ Rtrim :
  - ▣ All the characters are removed from right side as long as they appear in given set
  - ▣ Select ltrim('I like India','India') from dual
- ❑ Lpad :
  - ▣ Fills specified chars on left side of string
  - ▣ Select lpad('hello',10,'\*') from dual;
- ❑ Rpad :
  - ▣ Fills specified chars on right side of string
  - ▣ Select rpad('hello',10,'\*') from dual;

# General Functions

- ❑ The COALESCE() function returns the first non-null value in a list
- ❑ Example :
- ❑ `Select coalesce(null,3,1) from dual;`
  
- ❑ The DECODE() function allows you to add procedure if-then-else logic to queries.
- ❑ Example :
- ❑ `Select DECODE(1,1,'Equal');`

# General Functions

- ❑ CASE WHEN expression goes through conditions and returns a value when the first condition is met (like an if-then-else statement).
- ❑ So, once a condition is true, it will stop reading and return the result.
- ❑ If no conditions are true, it returns the value in the ELSE clause.
- ❑ If there is no ELSE part and no conditions are true, it returns NULL.
- ❑ Example :
- ❑ 

```
SELECT CustomerName
FROM Customers
(CASE
    WHEN City IS NULL THEN Country
    ELSE City
END);
```

# Date functions:

## □ Add months

- ▣ ADD\_MONTHS(x, y) returns the result of adding y months to x. If y is negative, then y months are subtracted from x.

- ▣ Example :-

- ▣ Adding 13 months:

- ▣ SQL> **SELECT** ADD\_MONTHS('01-JAN-2012', 13) **FROM**  
dual;

- ▣ ADD\_MONTH

- ▣ -----

- ▣ 01-FEB-13

## ❑ **LAST\_DAY** function

- ❑ LAST\_DAY(x) returns the last day of the month part of x.
- ❑ The following example displays the last date in January 2012:
- ❑ SQL> **SELECT** LAST\_DAY('01-JAN-2012') **FROM** dual;
- ❑ LAST\_DAY(  
❑ -----  
❑ 31-JAN-12  
❑

## □ Months\_between

- MONTHS\_BETWEEN(x,y) returns the number of months between x and y. If x is before y on the calendar, the number returned is positive; otherwise the number is negative.
- SQL> **SELECT** MONTHS\_BETWEEN('25-MAY-2009',  
'15-JAN-2012') **FROM** dual;
- MONTHS\_BETWEEN('25-MAY-2009','15-JAN-2012') -  
-----
- -31.677419

- ❑ **NEXT\_DAY function**

- ❑ NEXT\_DAY(x, day) returns the date of the next day following x. day is specified as a literal string (SATURDAY, for example).

- ❑ The following example displays the date of the next Saturday after January 1, 2008:

- ❑ SQL> **SELECT** NEXT\_DAY('01-JAN-2008', 'SATURDAY') **FROM** dual;

- ❑ NEXT\_DAY(

- ❑ -----

- ❑ 05-JAN-08

- ❑

## ❑ **Sysdate**

- ❑ SYSDATE returns the current date in the database server's operating system. The following example gets the current date:

- ❑ **SQL> SELECT SYSDATE FROM dual;**

- ❑ SYSDATE

- ❑ -----

- ❑ 14-JUL-11

- ❑

# Creating user & role

- ❑ In Oracle Database, you can create users and roles to manage access and permissions within the database.
- ❑ **Connect to Oracle Database:** You'll need administrative privileges or the necessary permissions to create users and roles.
- ❑ **Role:**
  - ❑ To create a role, use the **CREATE ROLE** statement.
  - ❑ Ex:-CREATE ROLE role\_name;
- ❑ **User:**
  - ❑ To create a user, use the **CREATE USER** statement.
  - ❑ Ex:-CREATE USER your\_username IDENTIFIED BY your\_password;

# **GRANTING AND REVOKING PERMISSIONS :**



## ■ What is Privileges ?

- ▣ The rights that allow the use of some or all of oracle's resources on the Server are called Privileges.

## ■ What is Granting ?

- ▣ When you create any object like table or etc you are owner of that object. So if you want other user use your object or they want to use your object you need to give them permission to access this is called granting of Privileges .

## ■ What is Revoking ?

- ▣ Privileges once given can be taken back by the owner of the object This is call Revoking of Privileges.

## □ Syntax of Granting permission using Grant statement :-

### □ Grant <object privileges>

- On <object name>

- To <UserName>

- [With Grant Option]

## □ The list of object privileges is as follow:

- Alter

- Delete

- Index

- Insert

- Update

- select

## □ Syntax of Revoking permission :

### □ Revoke <object privileges>

- On <object Name>

- From <UserName>

# Transaction :-

- What is transaction ?
  - ▣ A series of one or more SQL statements that are logically related on Oracle table data is termed as a Transaction.
  - ▣ A transaction will implicitly begin with as insert, update, delete, select....for update statement
  - ▣ Statement like COMMIT is used to ends the current transaction and makes permanently any changes.
  - ▣ Statement like ROLLBACK does exactly the opposite of commit. It ends the transaction but undoes any changes made during the transaction.

- 
- Statement like Savepoint marks and saves the current point in the processing of a transaction.
  - When savepoint use with a ROLLBACK statement, parts of a transaction can be undo

# CH-3

Other ORACLE Database Objects, Concurrency  
control using lock

VIEWS :

## ❑ What is view ?

- ❑ A view is a virtual table that stores only definition in the form of SQL statement. It is used to hide information from other user.

## ❑ Feature of view

- ❑ Creating view does not take any storage space in oracle
- ❑ The maximum number of columns that can be defined in a view is 1000
- ❑ When new records are inserted/updated/deleted in base table, the changes are automatically reflected in the view
- ❑ View once defined, can be used in Joins and Sub-query also

## ❑ Types of view

- ❑ Read only view
- ❑ Updateable view

# Syntax of view :-

- ❑ CREATE [OR REPLACE] [FORCE] VIEW <View Name> AS
- ❑ SELECT <Fields> FROM <Table> WHERE <condition>

SEQUENCE:-

# What is sequence :-

- ❑ A sequence is a set of integers that are generated in order on demand.
- ❑ Sequences are frequently used in databases because many applications require each row in a table to contain a unique value and sequences provide an easy way to generate them.
- ❑ Auto increment created in sequence so no need to remember field and insert data in that field with unique number



□ Sequences can be created, changed, and dropped with the following three SQL commands:

▣ CREATE SEQUENCE

▣ ALTER SEQUENCE

▣ DROP SEQUENCE

# Syntax:-

- ❑ Create sequence <name>
- ❑ [Start with <no>
- ❑ Increment by <no>
- ❑ Minvalue <no> / Nominvalue
- ❑ Maxvalue <no> / Nomaxvalue
- ❑ Cycle / Nocycle
- ❑ Cache / Nocache]

- A sequence definition may consist of a start value, increment value, minimum value, and maximum value.
- You can specify whether the sequence generator should stop when reaching a boundary value, or CYCLE the sequence numbers within the minimum/maximum range.
- All sequence attributes are optional and they all have default values.
- Each sequence has two columns: NEXTVAL and CURRVAL.
- NEXTVAL is use for nextvalue in sequence while CURRVAL use for current value of data

# Synonyms

- ❑ A synonym is an alternative name for objects such as tables, views, sequences, stored procedures, and other database objects.
- ❑ Synonyms is dependent object.
- ❑ **Syntax:**
- ❑ Create Synonym `Synonym_Name` for `Table_name`

# DATABASE LINKS :-



# Database links :-

- What is database link and use ?
  - ▣ As your data grow in size and number, you will very likely need to share data among them.
  - ▣ So sharing database to one location to another and manage data is call database link
  - ▣ For ex. If you create DB only for one college than its easy to access but when you have to handle all colleges of your city its quite difficult so you can share your database with other to maintain it

# Database links :-

- How database link works?
  - ▣ A database link requires that Net8 be running on each of the machines involved in the remote database access.
  - ▣ A database link establishes a connection from the first database to another
- Database specify following connection information
  - ▣ The communication protocol (tcp/ip) to use during the connection
  - ▣ The name of the database on the remote host
  - ▣ The password for that account when use and then logs out when remote data access is complete.

# INDEX

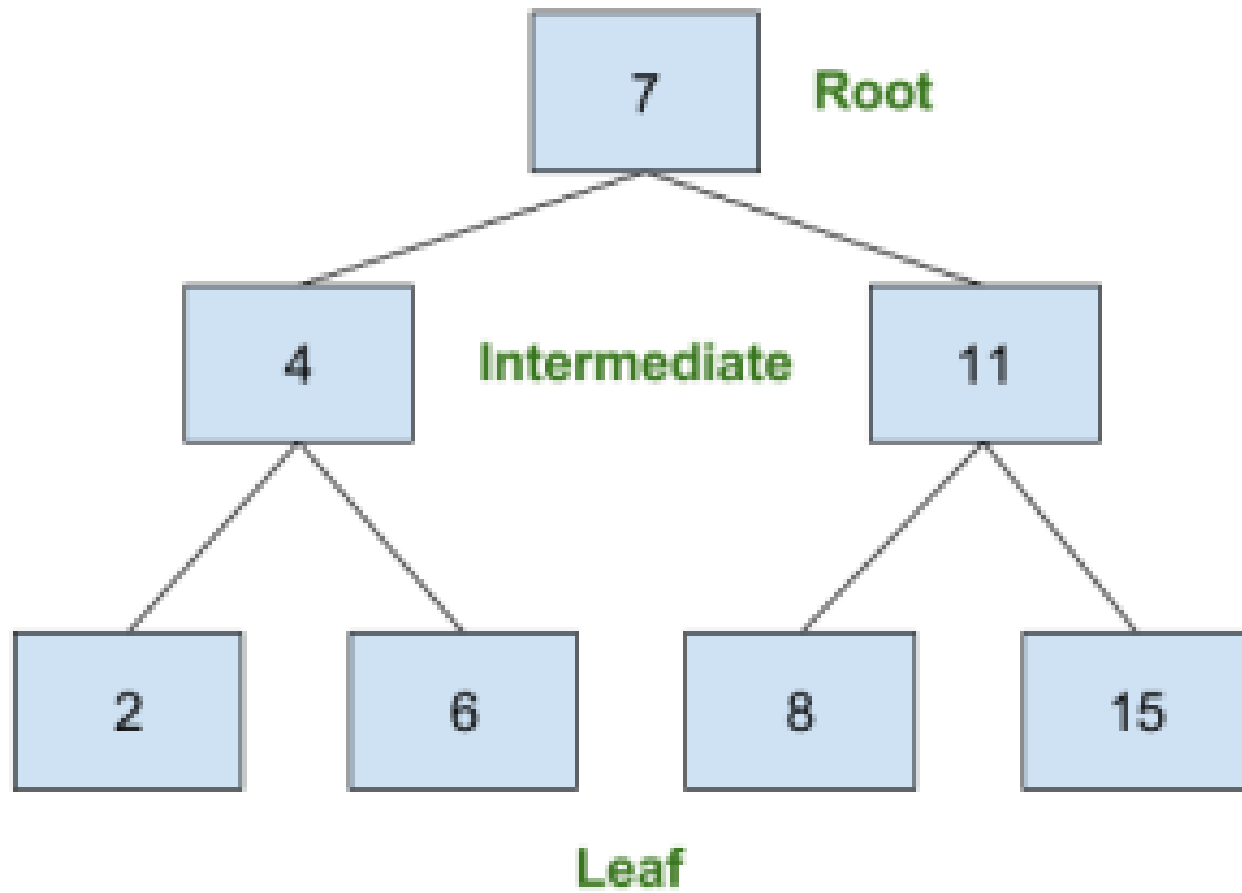


# What is indexes ?

- An **index** is a object that contains an entry for each value that appears in the **indexed** column(s) of the table and provides direct, fast access to rows.

# B tree index

- **B-tree** is a self-balancing **tree** data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in time.
- Each page in an **index B-tree** is called an **index** node. The top node of the **B-tree** is called the root node. The bottom nodes in the **index** are called the leaf nodes



# What is Bitmap indexes?

- ❑ A bitmap is a mapping from one system such as integers to bits. It is also known as bitmap index or a bit array.
- ❑ The memory is divided into units for bitmap.
- ❑ These units may range from a few bytes to several kilobytes.
- ❑ Each memory unit is associated with a bit in the bitmap.
- ❑ If the unit is occupied, the bit is 1 and if it is empty, the bit is zero.
- ❑ The bitmap provides a relatively easy way to keep track of memory as the size of the bitmap is only dependent on the size of the memory and the size of the units.

## Bitmap Indices Example

| record number | <i>ID</i> | <i>gender</i> | <i>income_level</i> |
|---------------|-----------|---------------|---------------------|
| 0             | 76766     | m             | L1                  |
| 1             | 22222     | f             | L2                  |
| 2             | 12121     | f             | L1                  |
| 3             | 15151     | m             | L4                  |
| 4             | 58583     | f             | L3                  |

Bitmaps for *gender*

m

10010

f

01101

Bitmaps for  
*income\_level*

L1

10100

L2

01000

L3

00001

L4

00010

L5

00000

# Functional based index

- ❑ **Function-based indexes** allow you to create an **index based** on a **function** or expression.
- ❑ Function-based indexes can involve multiple columns, arithmetic expressions, or maybe a PL/SQL function
- ❑ The following example shows how to create a function-based index:
  - ▣ Create index emp\_idx on emp (upper(ename));

# Application Domain Indexes:

- Application domain indexes are custom indexes that are specific to the needs of your application.
- They can be based on the semantics of your data and queries.
- Example:
- Suppose you have a table `books` with a `publication_date` column, and you want to create an index that takes into account the availability of books in a specific library branch.
- You could create an application domain index like this:
- `CREATE INDEX idx_library_availability ON books (branch_id, publication_date);`

# SQL CLUSTERS



- ❑ Clusters in SQL are used to store data that is from different tables in the same physical data blocks.
- ❑ They are used if records from those tables are frequently queried together.
- ❑ By storing same data blocks, the number of database block reads needed to full fill such queries decreases which improves performance.
- ❑ Each cluster stores tables data and maintains a clustered index to sort data.
- ❑ Columns within the cluster index are called clustered keys. These determine the physical placement of rows within the cluster.
- ❑ Cluster key is usually a foreign key of one table that references the primary key of another table in cluster.

- ❑ The first cluster is created. Then cluster index is created on cluster key columns.
- ❑ After cluster key index is created, data can be entered into tables stored in clusters.
- ❑ As rows are inserted database stores cluster key and its associated rows in each of the cluster's blocks.
- ❑ **Syntax :**
- ❑ CREATE CLUSTER <Cluster Name>
- ❑ (<Column> <Data Type> [, <Column> <Data Type> ]  
.....) [<Other Options >]

# Example :-

- ❑ create table branch\_master
- ❑ ( "branch\_code" varchar(10) PRIMARY KEY,  
"branch\_name" varchar(15) );
- ❑ Now, Consider the above created table and let's create the cluster for the same.

## **Creating a Cluster :**

- ❑ create cluster branch\_info
- ❑ ( "branch\_code" varchar(10) );

# Advantages of clusters :

---

- ❑ Disk I/O is reduced.
- ❑ Access time improves for joins of clustered tables.
- ❑ As all rows in clustered tables use the same columns as common primary key, this yields storage benefit.

# Disadvantages of clusters :

---

- ❑ Reduces performance of INSERT statements as compared to storing the table separately with its own index.
- ❑ Columns that are often updated are not good candidates for cluster key.

# SQL SNAPSHOTS



- ❑ **Snapshot** is a recent copy of the table from the database or a subset of rows/columns of a table.
- ❑ The SQL statement that creates and subsequently maintains a snapshot normally reads data from the database residing server.
- ❑ A snapshot is created on the destination system with the create snapshot SQL command. The remote table is immediately defined and populated from the master table.
- ❑ These are used to dynamically replicate data between distributed databases. Two types of snapshots are available.
- ❑ Simple snapshots
- ❑ Complex snapshots

# Simple snapshot :

- ❑ In **simple snapshot**, each row is based on a single row in a single remote table.
- ❑ This consists of either a single table or a simple SELECT of rows from a single table.
- ❑ **Example –**
- ❑ `CREATE SNAPSHOT emp_snap as select * from emp;`

# Complex snapshot :

- ❑ In **complex snapshot**, a row may be based on more than one row in a remote table via GROUP BY operation or result of Multi-Table Join.
- ❑ This consists of joined tables, views, or grouped and complex SELECT statement queries.
- ❑ **Example –**
- ❑ `CREATE SNAPSHOT sampleSnps1 AS SELECT student.rollno, student.name FROM student UNION ALL SELECT new_student.rollno, new_student.name FROM new_student;`

# Advantages :

---

- ❑ Response time is improved when local read-only copy of table exists.
- ❑ Ease network loads.
- ❑ Data subsetting.
- ❑ Disconnected computing.
- ❑ Mass deployment.

# Disadvantages :

---

- ❑ Snapshots are not reachable when primary database goes offline.
- ❑ It does not support full text indexing.
- ❑ Snapshot runs out of disk if data changes frequently faster.
- ❑ As no.of snapshots increases, disk space becomes problematic.

# LOCKS AND LATCHES



# What is locks?

---

- In Oracle PL/SQL, a **LOCK** is a mechanism that prevents destructive interaction between two simultaneous transactions or sessions trying to access the same database object

# Modes of Locking

## ❑ **Exclusive lock**

The mode prevents the associated resource from being shared

## ❑ **Share lock**

The mode allows the associated resource to be shared, depending on the operations involved

# Locking Issues

- ❑ Lost Updates
- ❑ o Pessimistic Locking
- ❑ o Optimistic Locking
- ❑ o Blocking
- ❑ o Deadlocks
- ❑ o Lock Escalation

# Lost Updates :

- ❑ A lost update problem occurs due to the update of the same record by two different transactions at the same time.
- ❑ In simple words, when two transactions are updating the same record at the same time in a DBMS then a lost update problem occurs

# Optimistic and Pessimistic lock

- ❑ Optimistic **locking** is when you check if the record was updated by someone else before you commit the transaction.
- ❑ **Pessimistic locking** is when you take an exclusive **lock** so that no one else can start modifying the record

# Blocking issue

- ❑ Blocking issues in Oracle occur when a session holds an exclusive lock on an object and doesn't release it before another session wants to update the same data.
- ❑ This blocks the second session until the first session has done its work.
- ❑ Blocking locks can happen when a session issues an insert, update, or delete command that changes a row.
- ❑ When the change occurs, the row is locked until the session either commits the change or rolls the change back.

# Deadlocks

- A deadlock occurs when two or more sessions are waiting for data locked by each other, resulting in all the sessions being blocked.
- Oracle automatically detects and resolves deadlocks by rolling back the statement associated with the transaction that detects the deadlock.

# Lock escalation :

---

- ❑ Lock escalation is a process that databases use to reduce the number of locks they hold.
- ❑ It occurs when a database raises the locks to a higher level on table.
- ❑ For example, if a single user locks many rows in a table, some databases automatically escalate the user's row locks to a single table.

# Types of Locks

| Lock                                | Description                                                                                             |
|-------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>DML locks</b> (data locks)       | DML locks protect data<br><br>For example, table locks lock entire tables, rowlocks lock selected rows. |
| <b>DDL locks</b> (dictionary locks) | DDL locks protect the structure of schema objects                                                       |
| <b>Internal locks</b> and latches   | Internal locks and latches protect internal database structures such as datafiles                       |

# CH-4

Introduction to PL/SQL, Advanced PL/SQL

# SQL v/s PL/SQL

| SQL                                                           | PL/SQL                                                                                           |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| It is a database Structured Query Language.                   | It is a database programming language using SQL.                                                 |
| Variables are not available in SQL.                           | Variables, constraints, and data types features are available in PL/SQL.                         |
| No Supported Control Structures like for loop, if, and other. | Control Structures are available like, for loop, while loop, if, and other.                      |
| It is a Data-oriented language.                               | It is an application-oriented language.                                                          |
| Query performs the single operation in SQL.                   | PL/SQL block performs Group of Operation as a single block resulting in reduced network traffic. |
| SQL is a declarative language.                                | PL/SQL is a procedural language.                                                                 |
| SQL can be embedded in PL/SQL.                                | PL/SQL can't be embedded in SQL.                                                                 |
| It directly interacts with the database server.               | It does not interact directly with the database server.                                          |
| SQL does not provide error and exception handling.            | PL/SQL provides error and exception handling.                                                    |

# PL SQL BLOCK STRUCTURE



# PL/SQL block structure :-

- Any language is useless if it has no programming power.
- By writing simple SQL statement, we cannot fulfill all our requirements.
- As result PL/SQL is required for the purpose of programming.
- PL/SQL stands for Procedural Language/SQL
- PL/SQL is bloc structure language. Each block divided into three sections
  - ▣ Declaration Section (Optional)
  - ▣ Execution Section
  - ▣ Exception Section (Optional)

# Declaration Section :-

- ❑ This section starts with the keyword DECLARE
- ❑ Variables and constants are declared, initialized within this section.
- ❑ You must have to declare variables and constants in a declarative block before using them in a procedural statement.
- ❑ Declare Variables and Assigning values: You can define variable name, data type of a variable and its size.
- ❑ Data type can be: CHAR, VARCHAR2, DATE, NUMBER, etc
- ❑ This section is optional if no variables or constraints to declare

# EXECUTABLE SECTION :-

- ❑ This section is mandatory in PL/SQL
- ❑ This section begins with keyword BEGIN
- ❑ This section can have another PL/SQL block inside it
- ❑ Execution Section contains SQL – PL/SQL executable statements
- ❑ This section contains conditional statements (if...else), looping statements (for, while) and Branching Statements (goto) etc.

# EXCEPTION SECTION :-

- ❑ This is an Optional section
- ❑ This section start with keyword EXCEPTION
- ❑ PL/SQL easily detects user defined or predefined error condition.
- ❑ PL/SQL is famous for handling errors in smart way by giving suitable user friendly messages.
- ❑ Errors can be rise due to wrong syntax, bad logical or not passing a validation rules.

# Plsql block structure syntax :-

## □ Syntax

- DECLARE

- <variable / constant declaration>

- BEGIN

- <statements>

- [EXCEPTION

- <statements>]

- END;

# Language construct of PL/SQL

- ❑ PL/SQL is a powerful programming language extension for Oracle Database that is used to create and manage database applications.
- ❑ It includes a variety of language constructs for working with variables and data types.
- ❑ **Variables:**
- ❑ **Declaration:** In PL/SQL, you declare variables using the **DECLARE** keyword within a block or subprogram.
- ❑ Ex:-
- ❑ DECLARE  
my\_variable NUMBER;
- ❑ **Assignment:** You can assign values to variables using the **':='** operator
- ❑ Ex:-
- ❑ my\_variable := 42;

# Language construct of PL/SQL

- ❑ **Data Types:** PL/SQL supports various data types for variables, including **NUMBER**, **VARCHAR2**, **DATE**, etc.
- ❑ Ex:-
- ❑ DECLARE  
emp\_name VARCHAR2(50);  
emp\_id NUMBER(5);

# CONDITIONAL STATEMENTS



# PL/SQL If :-

- ❑ PL/SQL supports the programming language features like conditional statements.
- ❑ Its programming constructs are similar to how you use in programming languages like Java and C++.
- ❑ **Syntax: (IF-THEN statement):**
  - ❑ IF condition
  - ❑ **THEN**
  - ❑ Statement: {It **is** executed **when** condition **is true**}
  - ❑ **END IF;**

# Syntax: (IF-THEN-ELSIF-ELSE statement):

- ❑ IF condition1 **THEN**
- ❑ {...statements **to execute when** condition1 is **TRUE**...}
- ❑ ELSIF condition2 **THEN**
- ❑ {...statements **to execute when** condition2 is **TRUE**...}
- ❑ **ELSE**
- ❑ {...statements **to execute when** both condition1 and condition2 are **FALSE**...}
- ❑ **END IF;**

# PL/SQL Case Statement :-

- ❑ The PL/SQL CASE statement facilitates you to execute a sequence of statements based on a selector.
- ❑ A selector can be anything such as variable, function or an expression that the CASE statement checks to a boolean value.
- ❑ The CASE statement works like the IF statement, only using the keyword WHEN.
- ❑ A CASE statement is evaluated from top to bottom.
- ❑ If it gets the condition TRUE, then the corresponding THEN clause is executed and the execution goes to the END CASE clause.

# Syntax for the CASE Statement:

- ❑ CASE [ expression ]
- ❑ **WHEN** condition\_1 **THEN**
  - ❑ result\_1
- ❑ **WHEN** condition\_2 **THEN**
  - ❑ result\_2
- ❑ ...
- ❑ **WHEN** condition\_n **THEN**
  - ❑ result\_n
- ❑ **ELSE** result
- ❑ **END CASE;**
- ❑ **END;**

# LOOPS :-

# Basic loop:- infinite loop

- ❑ Basic loop structure encloses sequence of statements in between the **LOOP** and **END LOOP** statements.
- ❑ With each iteration, the sequence of statements is executed and then control resumes at the top of the loop.
- ❑ An **EXIT statement** or an **EXIT WHEN statement** is required to break the loop.

# Syntax of basic loop :-

## Syntax basic loop

- Syntax
  - ▣ LOOP
    - <STATEMENT>
    - [EXITWHEN(CONDITIO  
N)];
- END LOOP

## EXAMPLE OF BASIC LOOP

- DECLARE
- i NUMBER(3);
- BEGIN
- i := 1;
- LOOP
- DBMS\_OUTPUT.PUT\_LINE(i);
- i := i + 1;
- exit when i>10;
- END LOOP;
- END;
- /

# For loop :-

- ❑ A **FOR LOOP** is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.
- ❑ Syntax
- ❑ `FOR<VARIABLE>IN[REVERSE]START_RANGE..END_RANGE`
- ❑ `LOOP`
  - ▣ `<LOOP BODY STATEMENT> ;`
- ❑ `END LOOP ;`

- ❑ Following is the flow of control in a **For Loop** –
- ❑ The initial step is executed first, and only once. This step allows you to declare and initialize any loop control variables.
- ❑ Next, the condition, i.e., *initial\_value* .. *final\_value* is evaluated. If it is TRUE, the body of the loop is executed. If it is FALSE, the body of the loop does not execute and the flow of control jumps to the next statement just after the for loop.
- ❑ After the body of the for loop executes, the value of the counter variable is increased or decreased.
- ❑ You can reverse LOOP by using the **REVERSE** keyword

# Example of For loop :-

- ❑ DECLARE
- ❑     J NUMBER(2);
- ❑ BEGIN
- ❑     FOR J IN 1..10
- ❑     LOOP
- ❑         DBMS\_OUTPUT.PUT\_LINE(J);
- ❑     END LOOP;
- ❑ END;
- ❑ /

# While loop :-

- ❑ A **WHILE LOOP** statement in PL/SQL programming language repeatedly executes a target statement as long as a given condition is true.
- ❑ SYNTAX
- ❑     **WHILE <CONDITION>**
- ❑     **LOOP**
- ❑         **<LOOP BODY STATEMENT>**
- ❑     **END LOOP;**

# Example of while loop :-

- ❑ DECLARE
- ❑       J NUMBER(3);
- ❑ BEGIN
- ❑       J :=1;
- ❑       WHILE J<=10
- ❑       LOOP
- ❑               DBMS\_OUTPUT.PUT\_LINE(J);
- ❑               J :=J + 1;
- ❑       END LOOP;
- ❑ END;
- ❑ /

%TYPE AND %ROWTYPE :-

# %type :-

- ❑ To declare the variable of the same data type as the data base column, we may use the %type declaration.
- ❑ This is called dynamic data type assignment
- ❑ This is important, as we need not change the program code if the data type of the table column is changed
- ❑ Changes in table data type direct reflected in the program
- ❑ Example % type
  - ▣ V\_salary EMP.SALARY%TYPE

# %ROWTYPE :-

- ❑ Using %ROWTYPE attributes, a record variable can be declared implicitly based on the structure of a table or query
- ❑ In implicitly defined record, we don't have to describe each field in the record definition
- ❑ It means Oracle selects data type automatically for declaring %ROWTYPE attribute
- ❑ Example :-
  - `Jsalry emp%ROWTYPE`

# CURSOR



# Cursor in oracle :-

- What is cursor ?
  - ▣ When you execute a SQL statement from PL/SQL ,the ORACLE assigns a private work area for that SQL statement is known as CURSOR.
- What is use of it ?
  - ▣ The PL/SQL is mechanism by which you can name that work area and manipulate the information within it. Data stored in the cursor known as Active Data Set (ADS)
- Types of cursors in oracle
  - ▣ There are two types of cursor available in oracle
    - Implicit cursor
    - Explicit cursor

# Implicit cursor :-

- The implicit cursors are automatically generated by Oracle while an SQL statement is executed.
- These are created by default to process the statements when DML statements like INSERT, UPDATE, DELETE etc. are executed.
- Attributes of both cursor
  - ▣ %ISOPEN – returns true if cursor open, false otherwise
  - ▣ %FOUND – returns true if record was fetched successfully, false otherwise
  - ▣ %NOTFOUND - returns true if record was not fetched successfully, false otherwise
  - ▣ %ROWCOUNT – returns number of records processed from the cursor

# Explicit cursor :-

## □ Explicit cursor

- ▣ A cursor can also be opened for processing data through a PL/SQL block such a user-defined cursor known as explicit cursor

## □ Attributes of both cursor

- ▣ %ISOPEN – returns true if cursor open, false otherwise
- ▣ %FOUND – returns true if record was fetched successfully, false otherwise
- ▣ %NOTFOUND - returns true if record was not fetched successfully, false otherwise
- ▣ %ROWCOUNT – returns number of records processed from the cursor

# Syntax and steps to remember

- ❑ The syntax for creating an explicit cursor is –
  - ❑ `CURSOR cursor_name IS select_statement;`
- ❑ Working with an explicit cursor includes the following steps –
  - ❑ Declaring the cursor for initializing the memory
  - ❑ Opening the cursor for allocating the memory
  - ❑ Fetching the cursor for retrieving the data
  - ❑ Closing the cursor to release the allocated memory

# Steps

## ❑ Declaring the Cursor

- ▣ Declaring the cursor defines the cursor with a name and the associated SELECT statement.
- ▣ For example
- ▣ `CURSOR c_customers IS SELECT id, name, address FROM customers;`

## ❑ Opening the cursor

- ▣ Opening the Cursor allocates the memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it.
- ▣ For example, we will open the above defined cursor as follows –
- ▣ `OPEN c_customers;`

# Continue...

## ❑ Fetching the Cursor

- ❑ Fetching the cursor involves accessing one row at a time. For example, we will fetch rows from the above-opened cursor as follows –

- ❑ `FETCH c_customers INTO c_id, c_name, c_addr;`

## ❑ Closing the cursor

- ❑ Closing the Cursor means releasing the allocated memory. For example, we will close the above-opened cursor as follows –

- ❑ `CLOSE c_customers;`

# EXCEPTION HANDLING



- ❑ PL/SQL supports programmers to catch such conditions using **EXCEPTION** block in the program and an appropriate action is taken against the error condition.
- ❑ There are two types of exceptions –
  - ❑ System-defined exceptions
  - ❑ User-defined exceptions

# System defined Exception :

- ❑ No\_data\_found :-
- ❑ In This exception we need not declare exception, we can use directly in our exception part when in our pl/sql block no data match
- ❑ This exception generally use for SELECT statement, when no record match we can use this in our exception part

# User define exception :-

- ❑ To create user defined exception just follow this steps
  - ❑ Declacre exception in Declaration part
    - Example : `not_eligible exception;`
  - ❑ Raise Declared exception in Execution part
    - Example : `raise not_eligible;`
  - ❑ In last our Exception part just put your declared exception in between WHEN and THEN for proper message to our exception
    - Example : `when not_eligible then`

# PROCEDURE AND FUNCTION



# Creating and using procedure :

- ❑ PL/SQL block is not stored in database, its compiled every time when its executed.
- ❑ To remove these restrictions, Oracle provides facility to declare 'Name Block' like Procedures, Functions, Packages and Triggers.
- ❑ The named block can be stored in the database and can be called and executed by client applications at any time

# Procedure and functions (subprograms)

- ❑ Procedures and functions are subprograms which store your PL/SQL block into database server of Oracle and executes when it needed.
- ❑ Functions take zero or more parameters and return a value.
- ❑ Procedures take zero or more parameters and return no values.
- ❑ The primary difference between procedures and functions is how they are called.
- ❑ Procedures are called as stand alone executable statements

# Parameter in procedure :-

- You may pick one of the following modes for each parameter:
- **IN** is the default mode for a parameter.
- IN parameters already have a value when the procedure is run.
- The value of IN parameters may not be changed in the body.
- **OUT** is specified for parameters whose values are only set in the body.
- **IN OUT** parameters may already have a value when the procedure is called, but their value may also be changed in the body.

# Syntax for procedure and function:

## □ PROCEDURE

- ▣ CREATE OR REPLACE PROCEDURE <procedure Name>
- ▣ (<Parameter list>) [IS/AS]
  - <Declaration Section>
- ▣ BEGIN
  - <PL/SQL Code>
- ▣ EXCEPTION
  - <PL/SQL Code>
- ▣ END;

## □ FUNCTION

- ▣ CREATE OR REPLACE FUNCTION <function name>
- ▣ (<Parameter List>) RETURN <Data Type> [IS/AS]
  - <Declaration Section>
- ▣ BEGIN
  - <PL/SQL Code>
  - RETURN <Expression>
- ▣ EXCEPTION
  - <PL/SQL Code>
- ▣ END;

PACKAGE :-

# PACKAGE :-

- A package is an object, which holds other objects like procedure, functions, cursor, exception etc. within it
  
- Components of Package :-
  - ▣ Package specification
  - ▣ Package body
  
- Package specification :-
  - ▣ Contains Name of the package
  - ▣ Contains declaration of Procedure, Function, Cursors, Exception etc.
  - ▣ The objects declared in a package header are visible outside the package
  - ▣ It does not contain any code for procedures / functions
  - ▣ Object must be declared in this section before it is referenced

# Syntax of specification of Package :-

- ❑ CREATE [OR REPLACE] PACKAGE <package name>
- ❑ {AS / IS }
  - ❑ Public\_Variable\_declarations
  - ❑ Public\_Exception\_declarations
  - ❑ Public\_Cursor\_declarations
  - ❑ Function\_specifications
  - ❑ Procedure\_specifications
- ❑ END [package name];

# Package Body :-

- ❑ Package body contains the definition of public objects that are declared in the specification
- ❑ Package body can also have other objects, which are private to the package
- ❑ If package header does not contain any procedure / functions then, package body is optional
- ❑ Private objects are defined in package body only and they are not visible / accessible outside the package.
- ❑ The variables declared in the package specification can be accessed by any procedure / function within the Package
- ❑ Package object referred as
  - EXECUTE <package Name>.<Object Name>
  - or
  - CALL <package Name>.<Object Name>

# PACKAGE BODY SYNTAX :-

- ❑ CREATE [OR REPLACE] PACKAGE BODY <package name>
- ❑ {AS / IS }
  - ❑ Private\_Variable\_declarations
  - ❑ Private\_Exception\_declarations
  - ❑ Private\_Cursor\_declarations
  - ❑ Function\_specifications
  - ❑ Procedure\_specifications
- ❑ END [package name];

# TRIGGER



# TRIGGER :-

- ❑ Triggers are the programs that are executed automatically in response to a change in the database.
- ❑ Oracle allows special types of procedures that are automatically executed when events like INSERT/ UPDATE/ DELETE occurs. These event procedures called “DATABASE TRIGGERS”
- ❑ You may not use COMMIT, ROLLBACK and SAVEPOINT statement within trigger blocks.
- ❑ Types of triggers
  - ▣ Before update statement level
  - ▣ After insert row level
  - ▣ Before delete row level

# Types of Trigger :-

- ❑ Before update statement level
  - ▣ In this type, trigger fires when any kind of updates occur
  - ▣ For ex. If you write query of update and this type of trigger fired on your table the changes occur before update any data
- ❑ After insert row level
  - ▣ In this type, trigger fires when row insert in database table
  - ▣ For ex. If you insert any data in your database table and trigger is fired in your table then after inserting row trigger run
- ❑ Before delete row level
  - ▣ In this type, trigger fires before delete any record in table row
  - ▣ For ex. If you are delete any records in your database table this trigger fires automatically an make changes as per trigger programmed

# SYNTAX OF TRIGGER :-

- ❑ CREATE [OR REPLACE] TRIGGER <name>
- ❑ [BEFORE / AFTER / INSTEAD OF]
- ❑ [DELETE / INSERT / UPDATE [OF <col1>,<col2>..]]
- ❑ ON <table / view name>
- ❑ [FOR EACH ROW [WHEN <condition>]]
- ❑ DECLARE
  - ❑ <VAR>
- ❑ BEGIN
  - ❑ PL/SQL STATEMENT
- ❑ EXCEPTION
  - ❑ EXCEPTION
- ❑ END;

# TRIGGER PREDICATES :-

- When the trigger is created for more than one DML operation, one can use conditional predicates within the trigger body to execute specific blocks of code, depending on the types of statement that fires the trigger.

| Predicate | Behavior                                      |
|-----------|-----------------------------------------------|
|           |                                               |
| INSERTING | TRUE if the triggering statement is an INSERT |
| DELETING  | TRUE if the triggering statement is an DELETE |
| UPDATING  | TRUE if the triggering statement is an UPDATE |

# :OLD AND :NEW

- The :new refers to a record containing the new, post change data.
- The :old refers to a record containing the old, pre-change data.
- The trigger can have visibility to both old and new values.
- :NEW can be modified only in a before row-level trigger, and :OLD can never be modified but only read from

| statement | :OLD                                      | :NEW                                                           |
|-----------|-------------------------------------------|----------------------------------------------------------------|
| INSERT    | Undefined – all fields are null           | Values that will be inserted when the statement is complete    |
| UPDATE    | Original values for the row before update | New values that will be updated when the statement is complete |
| DELETE    | Original values before the row is deleted | Undefined – All fields are null                                |

# Creating Objects in Oracle:

- ❑ **Object Types:** you can create custom data structures in Oracle using object types.
- ❑ Object types allow you to define your own structured data types, complete with attributes and methods.
- ❑ **Ex:-**
- ❑ 

```
CREATE OR REPLACE TYPE AddressType AS OBJECT (  
    street VARCHAR2(50),  
    city VARCHAR2(50),  
    postal_code VARCHAR2(10)  
);
```

# Object in Database Table:

- Once you've defined an object type, you can use it as a column data type in a database table, effectively creating an object-relational mapping.
- **Ex:-**
- ```
CREATE TABLE Employee (  
    emp_id NUMBER,  
    emp_name VARCHAR2(50),  
    emp_address AddressType  
);
```

# PL/SQL Tables:

- PL/SQL tables are indexed collections, which are similar to arrays in other programming languages. They are mainly used to hold sets of data temporarily.
- PL/SQL tables can be either associative arrays (index-by tables) or nested tables.

- **Ex:-**

- DECLARE

```
TYPE StringArray IS TABLE OF VARCHAR2(50) INDEX BY PLS_INTEGER;  
names StringArray;
```

```
BEGIN
```

```
names(1) := 'Alice';
```

```
names(2) := 'Bob';
```

```
names(3) := 'Charlie';
```

```
END;
```

# Nested Tables:

- ❑ Nested tables are similar to PL/SQL tables, but they can be stored in the database as a column in a table or as a database object.
- ❑ Nested tables can be considered as unbounded arrays.
- ❑ **Ex:-**
- ❑ 

```
CREATE TABLE Orders (  
    order_id NUMBER PRIMARY KEY,  
    products NestedTableType)  
NESTED TABLE products STORE AS ProductsTable;
```

# VARRAY



- 
- ❑ The varray (variable size array) is one of the three types of collections in PL/SQL (associative array, nested table, varray).
  - ❑ The varray's key feature is that when you declare a varray type, you specify the maximum number of elements that can be defined in the varray.

# Creating a Varray Type

- ❑ A varray type is created with the **CREATE TYPE** statement.
- ❑ The basic syntax for creating a VARRAY type
- ❑ CREATE OR REPLACE TYPE *varray\_type\_name* IS VARRAY(*n*) of *<element\_type>*
  - ▣ *varray\_type\_name* is a valid attribute name,
  - ▣ *n* is the number of elements (maximum) in the varray,
  - ▣ *element\_type* is the data type of the elements of the array.
- ❑ For example –
  - ▣ TYPE *namearray* IS VARRAY(5) OF VARCHAR2(10);
  - ▣ Type *grades* IS VARRAY(5) OF INTEGER;

# CH-5

Oracle Database Structure and Storage Database,  
Resource Management and Task Scheduling

# INSTANCE ARCHITECTURE :-



# Instance Architecture :-

- When a database is started on a database server, Oracle allocates a memory area called the System Global Area (SGA) and starts one or more oracle processes.
- This combination of the SGA and the oracle processes is called an Oracle Instance
- The instance and the Database :-
  - ▣ After starting an instance, Oracle associates the instance with the specified database. This is called Mounting the database.
  - ▣ The database then ready to be Opened, which makes it accessible to authorized users.
  - ▣ Only the database administrator can start up an instance and open the database.
  - ▣ If a database is open, then the database administrator can shut down the database so that its closed
  - ▣ Security for database startup and shutdown is controlled through connection to oracle with administrator privileges.
  - ▣ Normal users do not have control over the current status of an oracle database.

## □ Instance and database startup :-

- ▣ Start an instance
- ▣ Mount the database
- ▣ Open the database

## □ Database and instance shutdown :-

- ▣ Close the database
- ▣ Unmount the database
- ▣ Shut down the instance.

## □ An Oracle instance is build up by considering following aspects :-

- ▣ 1. Database Process
- ▣ 2. Memory Structure
- ▣ 3. Data Files

# Database Process :-

- The Oracle RDBMS uses two types of processes , user process and oracle process (called background process)
- User processes :
  - ▣ Its user's connection to the RDBMS system. It communicates user input with the Oracle server process-by using oracle program interface.
  - ▣ Its also used to display the information requested by the user.
- Oracle processes :-
  - ▣ It performs the function for users which divided into two processes :
    - A. server processes
    - B. Background processes

## □ Server process

- ▣ Oracle creates server process to handle the request of user processes. This server creates for the for the following
  - Run sql statements issued through the application
  - Read necessary data blocks from data files on disk
  - Return the result in such a way that the application can process the information

## □ Background process :-

- ▣ To maximize performance and accommodate many users, a multiprocess Oracle system uses some additional oracle processes called background processes.

## Background process in Oracle are follow :

- ❑ Database Writer Process (DBW)
- ❑ Log Writer Process (LGWR)
- ❑ Checkpoint Process (CKPT)
- ❑ System Monitor Process (SMON)
- ❑ Process Monitor Process (PMON)
- ❑ Archiver Processes (ARCn)
- ❑ Recoverer Process (RECO)
- ❑ Dispatcher
- ❑ Server

# Memory structure :-

- Oracle uses memory to store information such as following:
  - Program code
  - Information about connected session
  - Information needed during program execution
- Oracle memory structure have following attributes :
  - Database buffer cache
  - Redo log buffer
  - Shared pool
  - Dictionary cache

# Database buffer cache:-

- The database buffer cache is the portion of the SGA that holds copies of data blocks read from datafiles.
- The buffers in the cache are organized in two lists: the write list and the least recently used (LRU) list.
- The **write list** holds *dirty* buffers, which contain data that has been modified but has not yet been written to disk.
- The **LRU list** holds free buffers and dirty buffers that have not yet been moved to the write list.
- **Free buffers** do not contain any useful data and are available for use

# Redo Log Buffer

- ❑ The **redo log buffer** is a circular buffer in the SGA that holds information about changes made to the database.
- ❑ This information is stored in redo entries.
- ❑ **Redo entries** contain the information necessary to reconstruct changes made to the database by INSERT, UPDATE, DELETE, CREATE, ALTER, or DROP operations.
- ❑ Redo entries are used for database recovery, if necessary.

# Shared pool :-

- ❑ Shared pool has two cache
- ❑ Library cache
  - ▣ Executable code which connected to instance kept in library cache. Ex. Sql or pl/sql statements
- ❑ Dictionary cache
  - ▣ Dictionary cache is stored library of all oracle statements like select, insert, update etc so when user enter any query dictionary cache checks its syntax in library

# Data files :-

- The oracle architecture are the physical files where our information stored on disk.
- Oracle has several types for data files, each for different purpose :
  - ▣ Database data files
  - ▣ Control files
  - ▣ Online redo logs
  - ▣ Other database related files

# Database datafiles :

- Database data files are used to store data on disk.
- Database data files only written to by the Database writer processes
  - ▣ Tablespace :
    - Table space is logical storage unit within an Oracle database. Its logical because a tablespace is not visible in the file system of the machine on which the database stores
  - ▣ Create tablespace :
    - Create tablespace <tblsp\_name> '<datafile\_path>' Size<n M/K/G>
      - EXTENT MANAGEMENT LOCAL AUTOALLOCATE;
  - ▣ Alter tablespace :
    - ALTER TABLESPACE TABS1 ADD DATAFILE 'PATH' SIZE <N M/K/G>;
  - ▣ Drop tablespace :
    - DROP TABLESPACE TABSP1 INCLUDING CONTENTS;

## □ Control files :-

- ▣ The control file of database is binary file that contains a great deal of database information
- ▣ The control file contains name, data about the database log file.
- ▣ Oracle allows you to maintain duplicate copies with the help of control files.
- ▣ Its good if more then one copies stored in different disks to protect your database information.

## □ Online redo log file :-

- Online redo log file is like tape recorder that records every change in oracle in the oracle database.
- This recording is useful in system monitor because its also provide back up copy of your data to protect your information but it works online only
- The main advantage of online redo log file is when you lost your data you no need to request oracle DBA to rollback, redo log do automatically it.

## ❑ **Other database related files :-**

- ❑ some files which is useful in database related files as under

## ❑ **Oracle trace file :-**

- ❑ This file created by oracle in a number of different situation like database crash, a session failure, or based on specific user operational commands

## ❑ **Alert log file :-**

- ❑ This is general log file for each oracle database.

## ❑ **Networking configuration files :-**

- ❑ These files are used to configure the different network components of the oracle.

# ROLLBACK SEGMENT :-



# What is the Rollback Segment?

- ❑ In the ORACLE database, the rollback segment is a structure that stores previous data during transactions.
- ❑ To support the rollback facility we have a rollback segment in the ORACLE database.

# Uses of Rollback Segment:

- ❑ If during a transaction any failures occur then it makes sure that changes are not reflected back into our original database.
- ❑ It helps us in restoring our database in a consistent state in case of any failure. To ensure data consistency and isolation property.
- ❑ The rollback segment is used to recover our data in case of database failure.
- ❑ With the help of the Rollback command, we can restore our previous data which is stored inside the rollback segment to our original database.

# Rollback Segment :-

- When transaction begins, its assigned to a rollback segment. Rollback segments store the “before” image of data which is ready to run in oracle.
- When creating image of data some time more memory spend to data which is useless so rollback segment undo these extra memory space and keep data in their perfect size.
- Oracle 9i and above version allows you to undo space to be managed in either of two ways :-
  - Manually
  - Automatically

## □ Manually :-

- ▣ This mode is commonly referred to as rollback managed undo (RMU). Its invoke by setting the parameter `UNDO_MANAGEMENT` to `MANUAL`

## □ Automatically :-

- ▣ This mode is commonly referred to as automatically managed undo (AMU). Its invoke by setting the parameter `UNDO_MANAGEMENT` to `AUTO`.

## □ Create rollback segment :-

- ▣ `CREATE [PUBLIC] ROLLBACK SEGMENT rbs_name option(s)`

### ▣ OPTIONS:

- `TABSPACE tablespace_name`
- `STORAGE storage_clause`



□ Alter rollback segment :-

- ▣ ALTER ROLLBACK SEGMENT rbs\_name {ONLINE | OFFLINE}
- ▣ ALTER ROLLBACK SEGMENT rbs\_name STORAGE storage clause

□ DROP ROLLBACK SEGMENT :-

- ▣ DROP ROLLBACK SEGMENT rbs\_name

# How to CREATE/ALTER/DROP a Rollback Segment?

- ❑ As we know that Oracle database uses the rollback segment to recover our consistent data in case of any database failure be it hardware failure or software failure.
- ❑ **Create:** We can create a rollback segment by following way:
- ❑ ALTER ROLLBACK SEGMENT name\_of\_the\_segment [STORAGE Storage\_Clause] [ONLINE | OFFLINE] [SHRINK]**Drop:** Drop the rollback segment then we can do it in the following way:
- ❑ DROP ROLLBACK SEGMENT name\_of\_the\_segment

SQL \* LOADER

- ❑ What is sql loader do ?
- ❑ **SQL LOADER** utility is used to load data from other data source into Oracle.
- ❑ For example, if you have a table in FOXPRO, ACCESS or SYBASE or any other third party database, you can use **SQL Loader** to load the data into Oracle Tables.

# SQL\*Loader Features

- ❑ Load data across a network. This means that you can run the SQL\*Loader client on a different system from the one that is running the SQL\*Loader server.
- ❑ Load data from multiple datafiles during the same load session.
- ❑ Load data into multiple tables during the same load session.
- ❑ Specify the character set of the data.
- ❑ Selectively load data (you can load records based on the records' values).
- ❑ Manipulate the data before loading it, using SQL functions.
- ❑ Generate unique sequential key values in specified columns.
- ❑ Load data from disk

# **MANAGING AUTOMATED DATABASE MAINTENANCE TASKS IN ORACLE**



- 
- ❑ This is to refer to the process of using Oracle Database's built-in automation features and tools to efficiently and proactively maintain the health, performance, and reliability of an Oracle database.
  - ❑ Oracle provides a range of automated tasks and features to simplify routine maintenance activities that database administrators (DBAs) typically perform manually.
  - ❑ These tasks include:

- ❑ **Backup and Recovery:** Oracle offers tools like Recovery Manager (RMAN) to automate database backups, ensuring that data can be recovered in case of a failure.
- ❑ **Patching and Upgrades:** Oracle provides automated patching and upgrade solutions, such as Patch Set Updates (PSUs) and Critical Patch Updates (CPUs), to keep the database software up-to-date with the latest security and performance enhancements.
- ❑ **Statistics Gathering:** Regularly collecting and refreshing database statistics is crucial for query optimization. Oracle can automatically gather statistics on database objects.
- ❑ **Performance Monitoring and Tuning:** Oracle's Automated Database Diagnostic Monitor (ADDM) analyzes database performance and provides recommendations for improving it.
- ❑ **Space Management:** Automated features help manage tablespace, control file, and redo log file space.

- ❑ **Task Scheduling:** Oracle allows DBAs to schedule routine maintenance tasks, making it easier to execute them during non-peak hours.
- ❑ **Alerts and Notifications:** Automated systems can generate alerts and notifications when predefined conditions or thresholds are met, allowing for proactive problem resolution.
- ❑ **Resource Management:** Oracle Database can automatically allocate and manage system resources to optimize performance for various workloads.
- ❑ **Security Updates:** Automatic security updates help protect the database against vulnerabilities.

# MANAGING RESOURCES WITH ORACLE DATABASE RESOURCE MANAGER



- 
- Oracle Database Resource Manager is a powerful feature that allows database administrators (DBAs) to manage and priority base resource usage to ensure optimal performance and fairness among different database users and applications.

- **Resource Allocation:** Oracle Database Resource Manager enables DBAs to allocate system resources, such as CPU, memory, and I/O, to different user groups, applications, and database tasks based on predefined rules and priorities. This ensures that critical applications receive the necessary resources while preventing resource contention and performance bottlenecks.
- **Workload Management:** DBAs can define and manage resource consumer groups, each representing a category of users or applications with specific resource requirements. For example, a DBA can create separate consumer groups for online transaction processing (OLTP) and reporting workloads, assigning different resource shares to each group.

- **Resource Plans:** Resource Manager allows the creation of resource plans that specify how resources are distributed among consumer groups during different time intervals or specific events. For instance, during peak business hours, a resource plan may prioritize resources for the OLTP consumer group, ensuring responsive transaction processing.
- **Automatic Switching:** Resource Manager can automatically switch between resource plans based on predefined conditions or performance thresholds. This adaptive management ensures that resources are allocated appropriately as workload patterns change.

- **Priority-Based Queuing:** Resource Manager supports queue-based scheduling, allowing high-priority tasks to be executed before lower-priority ones. This helps ensure that mission-critical processes are not delayed by less important workloads.
- **Monitoring and Reporting:** Resource Manager provides monitoring tools and reports to help DBAs track resource usage and identify potential resource contention issues. It also offers insights into which consumer groups are utilizing resources most effectively.

# ORACLE SCHEDULER CONCEPTS

- 
- Oracle Scheduler Concepts refer to the fundamental ideas and components that form the basis of Oracle Scheduler, a powerful job scheduling and automation tool within the Oracle Database. Oracle Scheduler is designed to help automate and manage a wide range of tasks, jobs, and processes within the database environment.

- **Job:** A job is a specific task or unit of work that you want to automate using Oracle Scheduler. Jobs can include activities like running SQL scripts, PL/SQL procedures, external scripts, or data transformations. Jobs are at the core of Oracle Scheduler.
- **Program:** A program defines what a job does. It specifies the executable and arguments required to perform a particular task. Programs can include stored procedures, executable files, or scripts. A job references a program to execute a specific task.
- **Schedule:** A schedule defines when and how often a job should run. It specifies the frequency, start time, end time, and recurrence pattern for executing a job. Schedules allow you to control the timing of job execution.

- **Job Class:** Job classes are used to categorize and manage jobs. You can assign jobs to specific job classes based on their characteristics or priorities. Job classes help in resource allocation and prioritization of job execution
- **Window:** A window is a specific time interval during which jobs are allowed to run. Windows are often used to control when maintenance tasks or resource-intensive jobs can be executed, ensuring they don't interfere with critical workloads.
- **Dependency:** Dependency defines the relationship between jobs. You can specify that one job should only run after another job has completed successfully. Dependencies enable you to create job chains and manage complex workflows.

# Scheduling Jobs with Oracle Scheduler

- "Scheduling Jobs with Oracle Scheduler" involves using Oracle Scheduler, a feature within the Oracle Database, to automate and manage the execution of various tasks or jobs at specified times or under specific conditions.

- **Creating Jobs:** To schedule jobs with Oracle Scheduler, you first need to define the jobs you want to automate. Jobs can include tasks like running SQL scripts, PL/SQL procedures, or external scripts. These jobs are encapsulated as objects within the database.
- **Defining Programs:** For each job, you need to specify a program that defines what the job does. The program specifies the executable or procedure to run, as well as any required arguments or parameters. Programs can be created within the database or reference external executables.
- **Setting Schedules:** Once you have defined jobs and programs, you need to create schedules that determine when and how often these jobs should run. Schedules can be configured to execute jobs at specific times of the day, week, or month, and they can include options like specifying start and end times or recurring patterns.

- ❑ **Monitoring and Notifications:** Oracle Scheduler provides tools for monitoring the execution of scheduled jobs. You can view job status, logs, and performance metrics. Notifications can be configured to alert administrators when specific events or job outcomes occur.
- ❑ **Job History:** Oracle Scheduler maintains a history of job executions, allowing administrators to track the progress of jobs, view job status, and diagnose problems. This historical data can be valuable for auditing and troubleshooting purposes.
- ❑ **Security and Access Control:** Access control mechanisms ensure that only authorized users can create, modify, or execute scheduled jobs. This helps protect sensitive data and ensures that job scheduling is secure.

# Administering Oracle Schedule

- **Job Management:** Administrators are responsible for creating, modifying, and deleting jobs. This includes defining the job type (e.g., SQL script, PL/SQL procedure), setting job attributes, specifying the program to execute, and assigning the job to schedules.
- **Programs and Executables:** Administrators manage programs and executables associated with jobs. This involves defining the programs, specifying executable files or PL/SQL procedures, and configuring program attributes.
- **Schedules:** Administering Oracle Scheduler involves the creation, modification, and deletion of schedules. Administrators determine when jobs should run by defining schedules, specifying start and end times, and configuring recurring patterns.

- **Dependency Management:** Administrators establish dependencies between jobs when needed. Dependencies define the order in which jobs are executed and ensure that specific conditions are met before a job starts.
- **Window Configuration:** Windows, which specify time intervals during which jobs are allowed to run, are set up and managed by administrators. Windows help control job execution during different time frames, such as peak and off-peak hours.
- **Resource Allocation:** Resource plans and job classes are configured to manage resource allocation to jobs. Administrators ensure that critical jobs receive adequate system resources while preventing resource contention.



BEST OF LUCK FOR EXAM

**ORACLE BY JJ SIR**

**H. J. DOSHI IT COLLAGE JAMNAGAR**